Big Data visualization

Empirical Systems Engineering

2026. Fall

Dr. Imre Kocsis, kocsis.imre@vik.bme.hu

Budapest University of Technology and Economics
Department of Artificial Intelligence and Systems Engineering
ftsrg Research Group



Too many observations

The data frame: rows and columns

- One row per *observation* (a person, a trip, a log line), one column per *variable*
- The shared model of statistics (R, pandas), data science and BI (SQL tables)
- The same data can be stored *wide* or *narrow*

Wide: one row per server

a column per variable

server	cpu %	mem %	disk %
web-1	71	45	80
web-2	64	52	77
db-1	38	88	91

melt (unpivot) →

← *pivot*

Narrow (long): one row per measurement

variable names become data

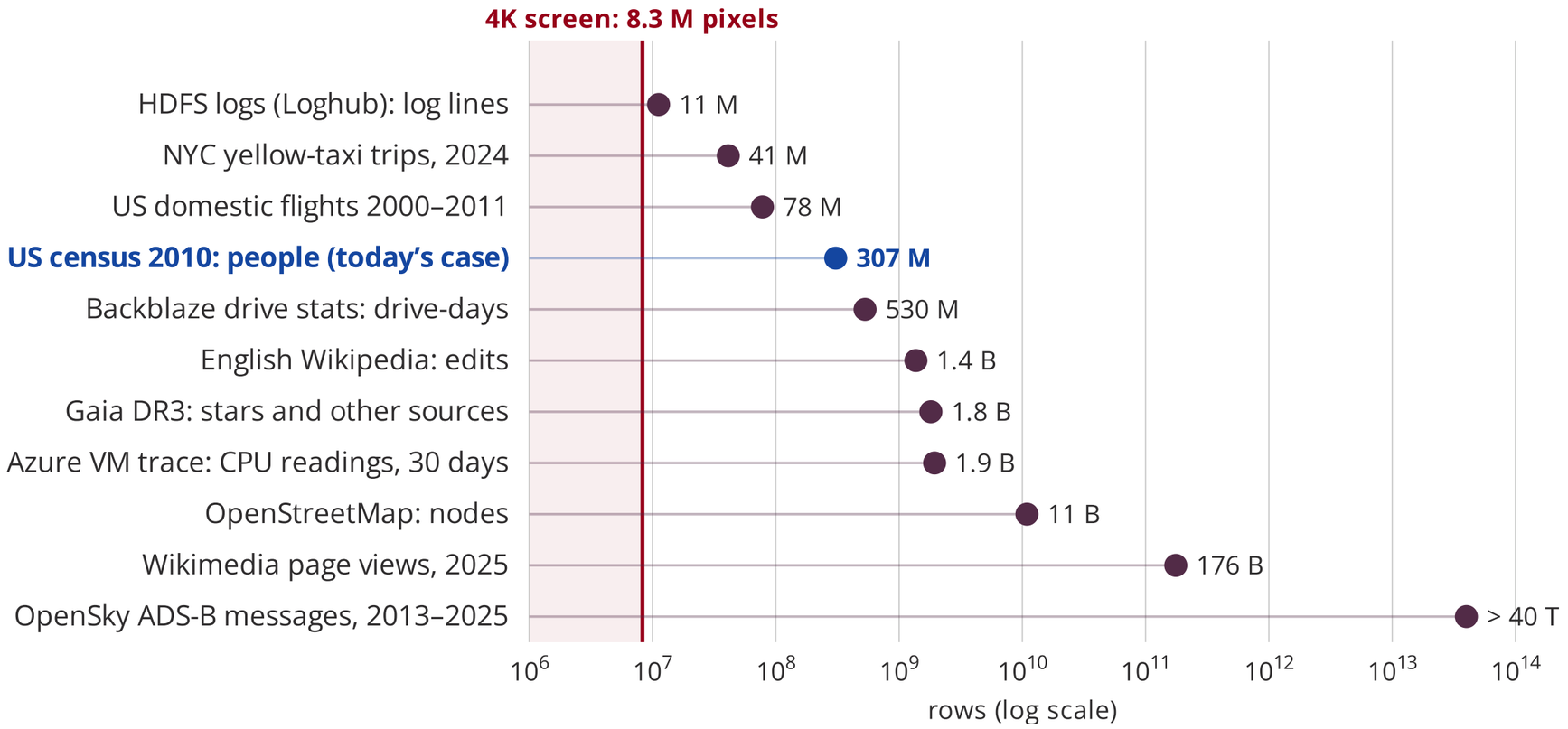
server	metric	value
web-1	cpu	71
web-1	mem	45
web-1	disk	80
web-2	cpu	64
web-2	mem	52
web-2	disk	77
db-1	cpu	38
db-1	…	…

grows in rows, fixed columns

H. Wickham: Tidy Data, Journal of Statistical Software 59(10), 2014

Too many rows, not too many columns

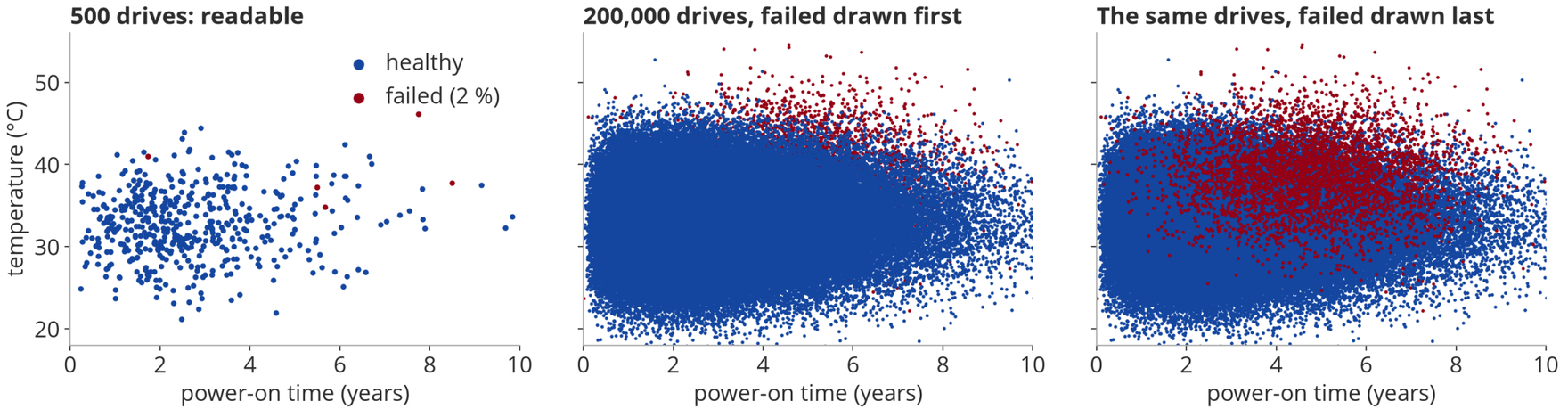
- This lecture: *too many observations*, from millions to trillions of rows
- “Too many variables” is a different topic



The pixel budget

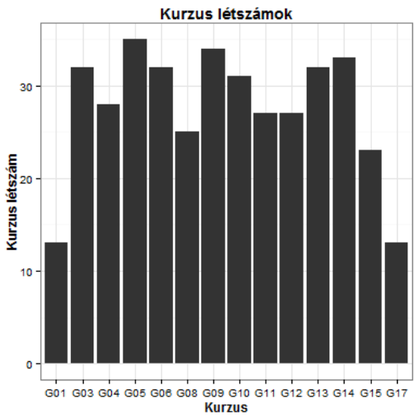
- Even 4K is only ≈ 8.3 M pixels
- A hard upper limit to laying out data
- No upper limit on data size
- Overplotting bites much earlier

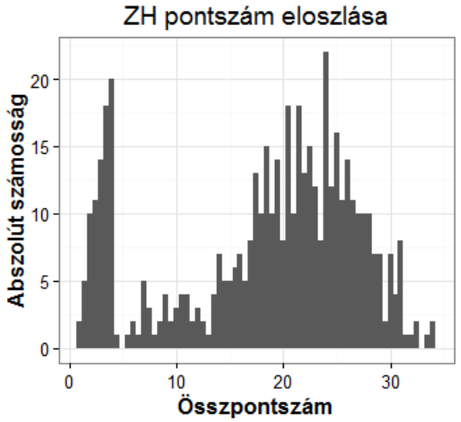


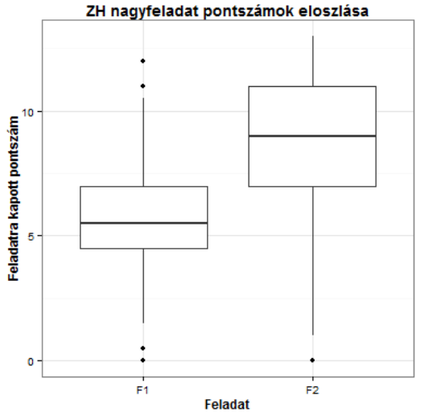


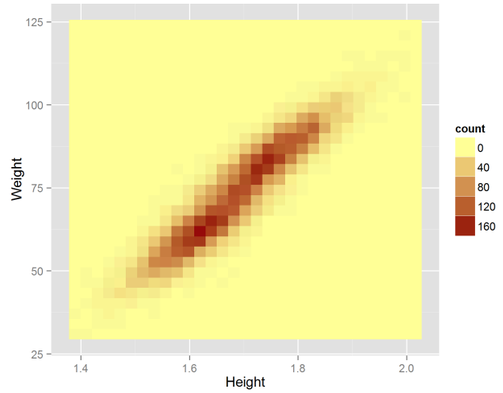
Simulated drive fleet, 2 % failed. With 200,000 drives the dots pile up, and whichever colour is drawn last wins.

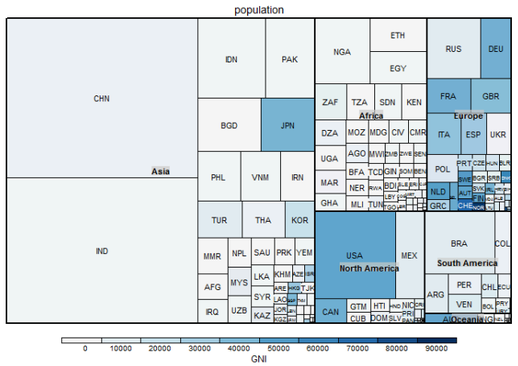
Which charts scale with the number of observations?

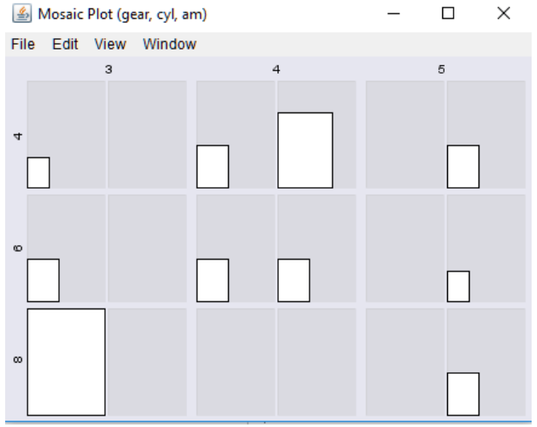


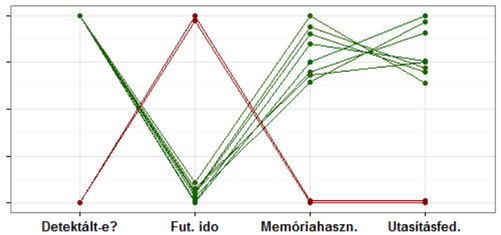


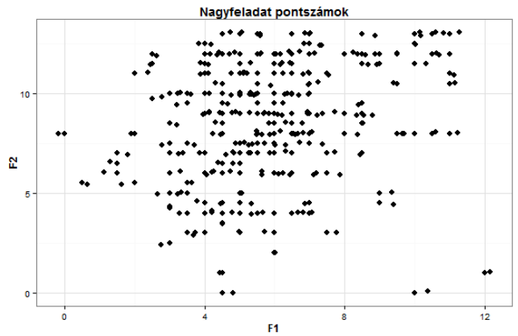






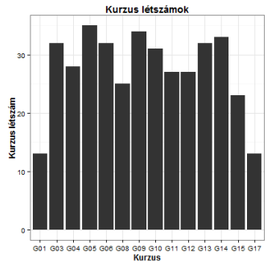




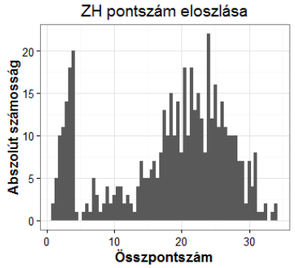


Aggregates scale, one-mark-per-row charts don’t

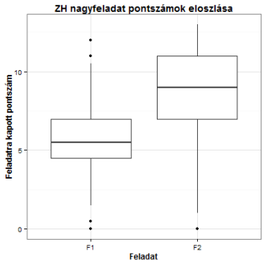
Scales: one mark per bin or category



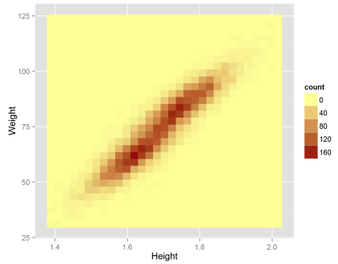
Bar chart of counts



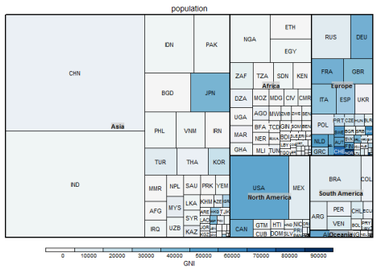
Histogram



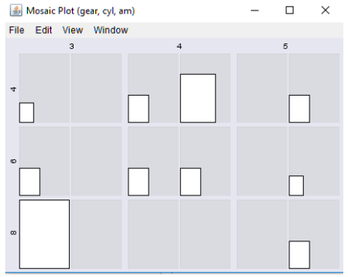
Boxplot



2D bins / heatmap

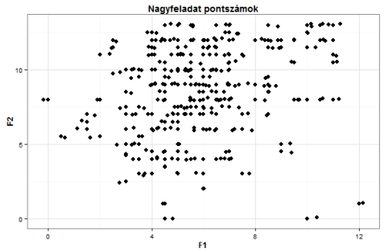


Treemap of totals

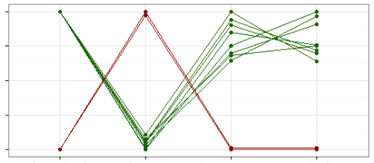


Mosaic plot

Doesn’t scale: one mark per observation



Scatter plot



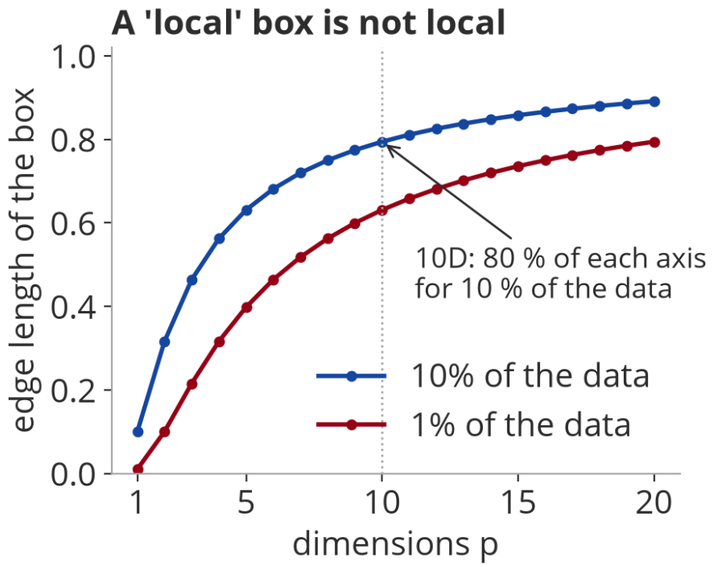
Parallel coordinates

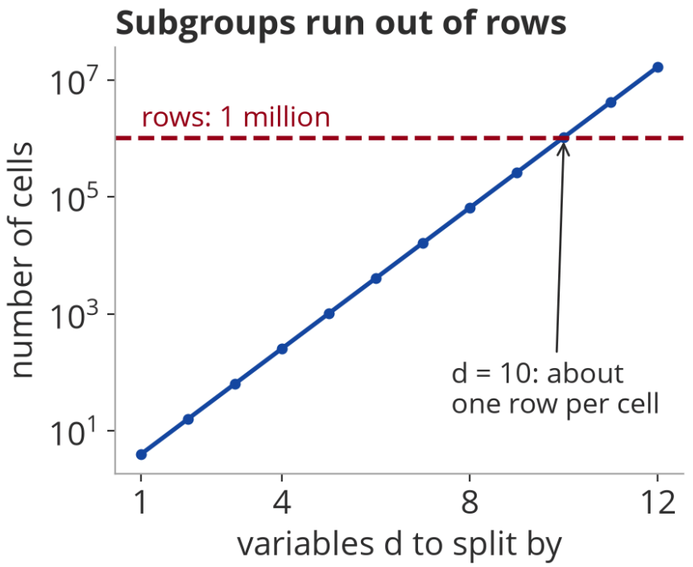
What works at scale

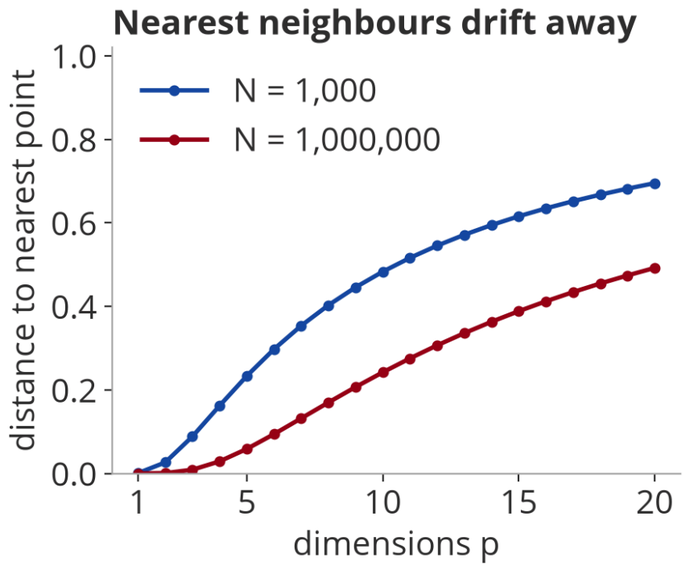
- (2D) EDA/CDA of (n, m)-dimensional statistical aggregates
- Subsetting: situational (may remain a Big Data problem)
- Sampling: hard to get right
- Much easier to “just process everything”

Sampling: the curse of dimensionality

Consistent coverage of the sample space as dimensions grow?







To achieve a fixed sampling density
Uniform data in a unit cube: a box holding a share r of it has edge $r^{1/p}$. In 10D, 10 % of the data spans 80 % of every axis. 100 points in 1D are as dense as 100^{10} in 10D.

To maintain statistical power in hypothesis testing
Split n rows by d variables of k levels: k^d cells of n/k^d rows. A cell mean's standard error grows as $k^{d/2}/\sqrt{n}$. An interaction needs 4× the rows of an equal main effect.

To keep a consistent distance between nearest-neighbour points
The nearest of N points lies about $N^{-1/p}$ away: halving it takes 2^p× more points. And distances concentrate: nearest and farthest become almost equally far.

Hastie et al.: Elements of Statistical Learning, §2.5 · Gelman, Hill, Vehtari: Regression and Other Stories, §16.4 · Beyer et al., ICDT 1999

Sampling: other traps

- Causal distortions: the return-visitor-ratio problem

  Sample 10 % of the *visits* instead of the visitors: a heavy user is almost surely in the sample, but with only a tenth of their visits. Per-visitor measures come out wrong. Fix: sample the unit you analyse (hash the user id, keep all their visits).

- Stratified sampling can help, if you know what to stratify on

  Split the population into strata known up front (country, device…) and sample each separately. It pays off when strata differ in mean or spread (Neyman allocation: $n_h \propto N_h S_h$). In exploration, the subgroups that matter are what you are looking for.

- Much easier to “just process everything”

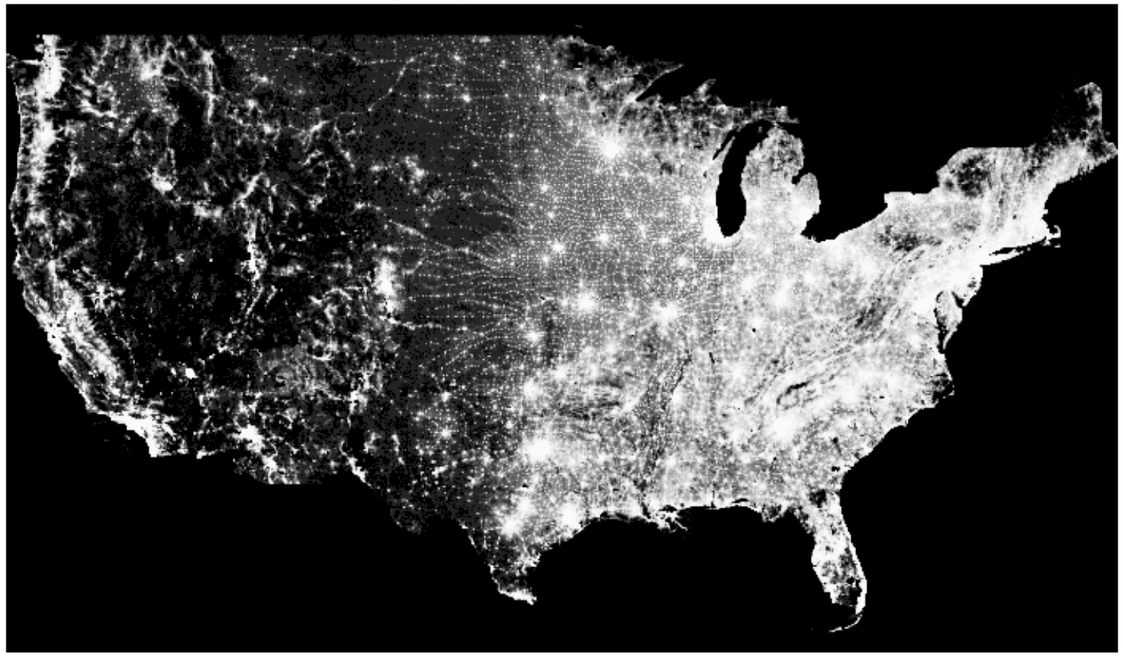
A toy web site: 60 % of users visit once, 30 % three times, 10 % ten times. Sample 10 % of the visits:

	all visits	**10 % sample**
returning users	40 %	17 %
visits per user	2.5	1.2
users with 10 visits	10 %	32 %

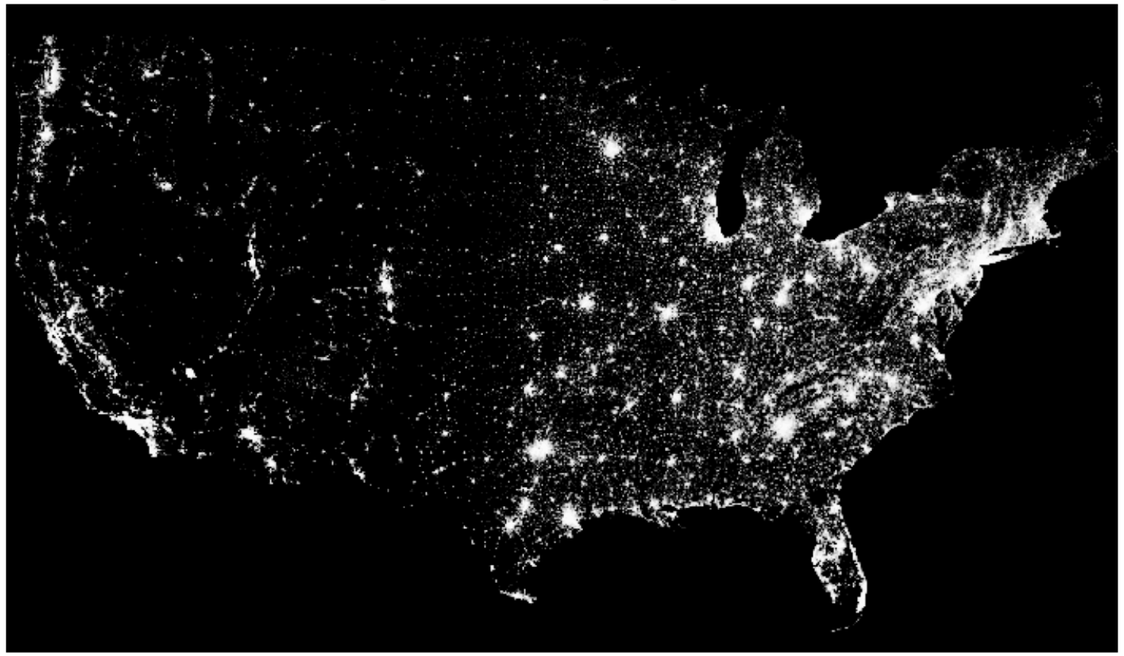
Deng, Knoblich, Lu: Applying the Delta Method in Metric Analytics, KDD 2018 · Cochran: Sampling Techniques, 3rd ed., 1977, ch. 5

Just process everything

All rows: 306,675,001 people



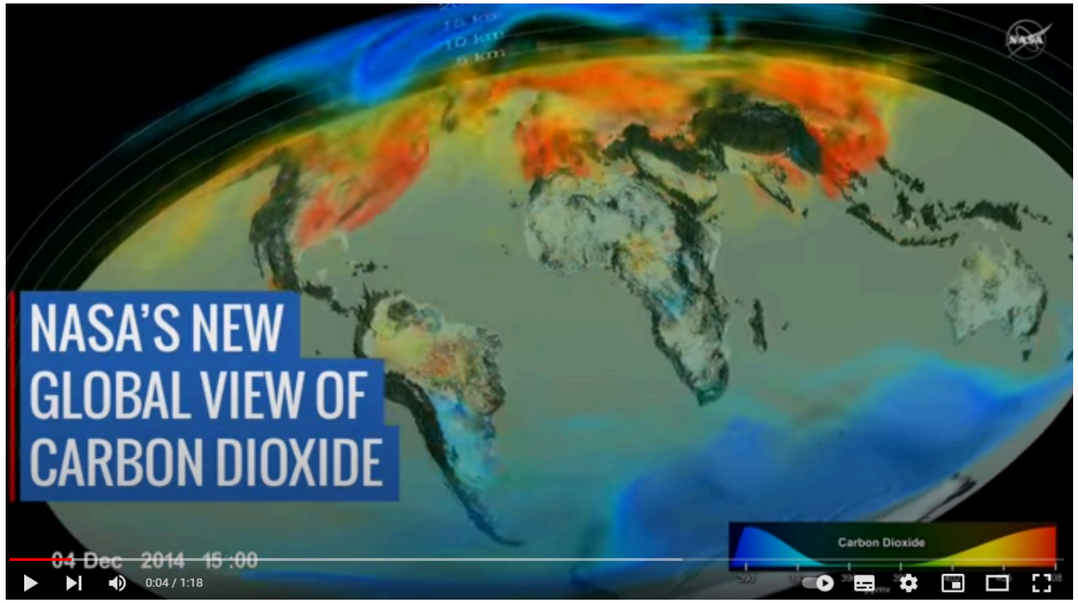
A 0.1 % random sample: 306,677 people



The census of the case study, same shading. A 0.1 % sample still has 306,677 rows, but the sparse West, the small towns of the Plains and the roads between them are gone: “the shape of an undersampled distribution can be very difficult or impossible to make out” (datashader’s plotting pitfalls). Aggregating every row costs one pass.

3D EDA: an aside

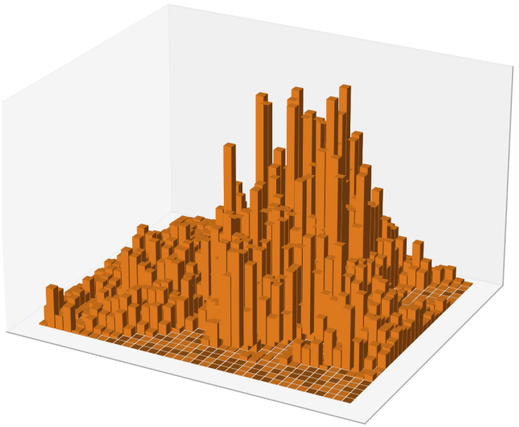
Sometimes it works…



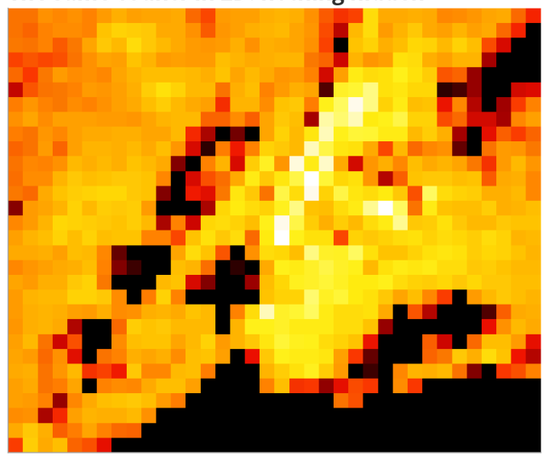
A year of CO_2 from NASA's GEOS model and OCO-2 satellite data: familiar geography, one variable in colour, time as motion.

…sometimes it doesn't

3D bars: tall ones hide what is behind them

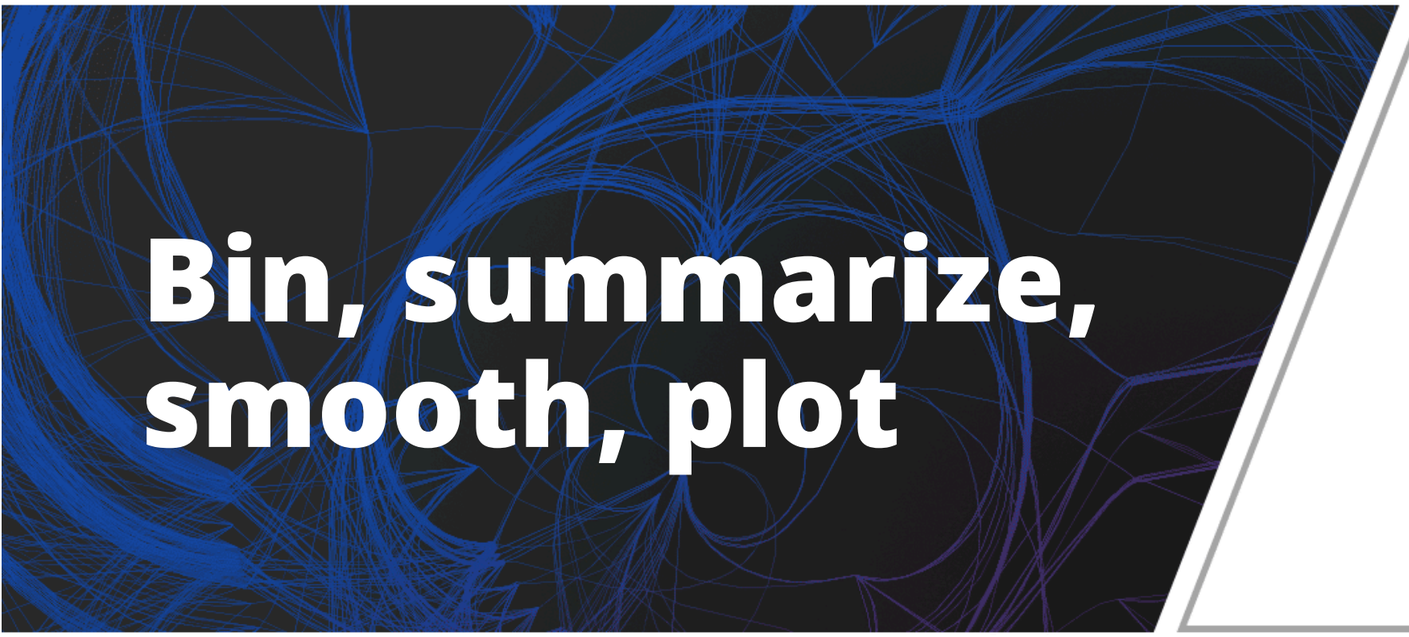


The same counts in 2D: nothing hidden



Population of New York City on a 36 × 30 grid (the case study's census). In 3D, Manhattan's towers hide what stands behind them, and heights are hard to compare in perspective.

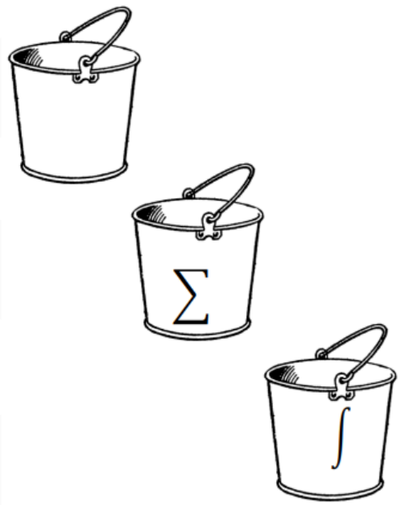
Video: NASA Goddard, Following Carbon Dioxide Through the Atmosphere (also on YouTube) · Right: census 2010, New York City

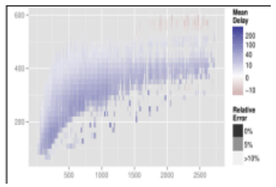


Bin, summarize, smooth, plot

The framework in four steps

- **Bin:** fixed-width bins over *n* dimensions
- **Summarize:** per-bin summaries over *m* dimensions
- **Smooth:** smooth, peel, transform distributions
- **Plot**





H. Wickham: Bin-summarise-smooth: a framework for visualising large data, tech report, 2013

Iris example: bin

Fisher's Iris data: 150 flowers, 3 species, 4 measurements in cm

#	sepal length	sepal width	petal length	petal width	species	bin
1	5.1	3.5	1.4	0.2	setosa	**0**
2	4.9	3.0	1.4	0.2	setosa	**0**
…						
51	7.0	3.2	4.7	1.4	versicolor	**7**
52	6.4	3.2	4.5	1.5	versicolor	**7**
…						
101	6.3	3.3	6.0	2.5	virginica	**10**
102	5.8	2.7	5.1	1.9	virginica	**8**
…						
150	5.9	3.0	5.1	1.8	virginica	**8**

Bin the petal length: 12 fixed-width bins of 0.5 cm, starting at 1.0 cm

$$b = \left\lfloor \frac{x - 1.0}{0.5} \right\rfloor$$

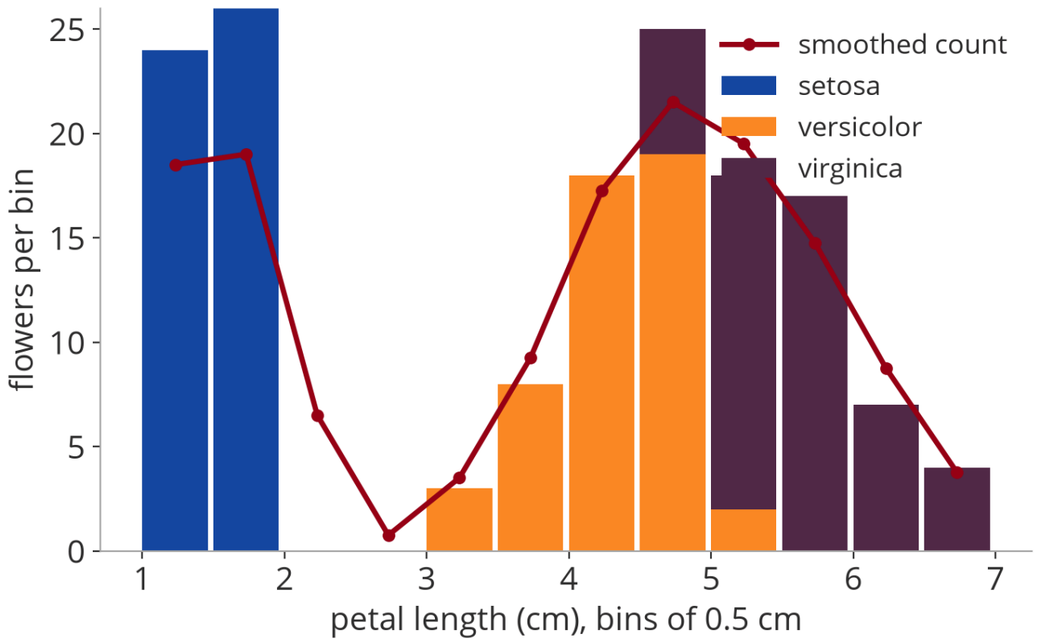
Flower 51: $\lfloor (4.7 - 1.0)/0.5 \rfloor = \lfloor 7.4 \rfloor = 7$

The bin is one more integer column, computed row by row: one pass, no data moves.

Iris data: R. A. Fisher, 1936 (as shipped with R)

Iris example: summarize, smooth, plot

bin	petal length	flowers	mean petal width	smoothed
0	1.0–1.5	24	0.22	18.5
1	1.5–2.0	26	0.27	19
2	2.0–2.5	0	–	6.5
3	2.5–3.0	0	–	0.75
4	3.0–3.5	3	1.03	3.5
5	3.5–4.0	8	1.14	9.25
6	4.0–4.5	18	1.27	17.25
7	4.5–5.0	25	1.55	21.5
8	5.0–5.5	18	1.97	19.5
9	5.5–6.0	17	2.05	14.75
10	6.0–6.5	7	2.11	8.75
11	6.5–7.0	4	2.15	3.75



Summarize: count (distributive), mean petal width (algebraic: carry *n* and Σ), count per species (like datashader's `by()`). **Smooth:** each bin gets ¼ · left + ½ · itself + ¼ · right. **Plot:** 12 bars, however many flowers.

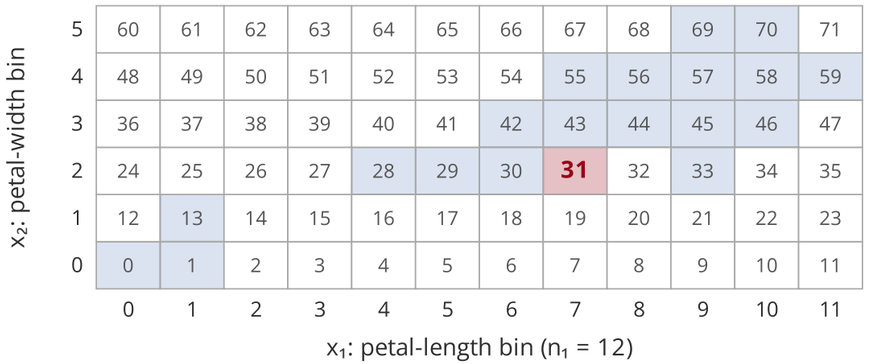
Binning is a logical operation

- A histogram, generalised: each bin is just an integer
- Computing it is “just a perfect hash”; no data moves
- *d* binned dimensions, bin x_j out of n_j in dimension j:

$$b = x_1 + x_2 n_1 + x_3 n_1 n_2 + \ldots + x_d \prod_{i=1}^{d-1} n_i$$
$$= x_1 + n_1 \left(x_2 + n_2 (x_3 + \ldots + n_{d-1} x_d) \right)$$

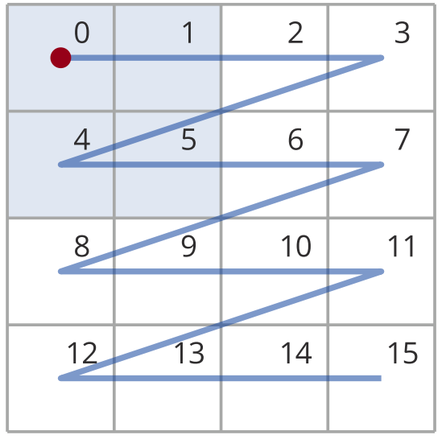
Example: Iris flower 51, petal 4.7 × 1.4 cm, bins of 0.5 cm

- length: $x_1 = \lfloor (4.7 - 1.0)/0.5 \rfloor = 7$ of $n_1 = 12$
- width: $x_2 = \lfloor 1.4/0.5 \rfloor = 2$ of $n_2 = 6$
- $b = x_1 + x_2 n_1 = 7 + 2 \cdot 12 = 31$

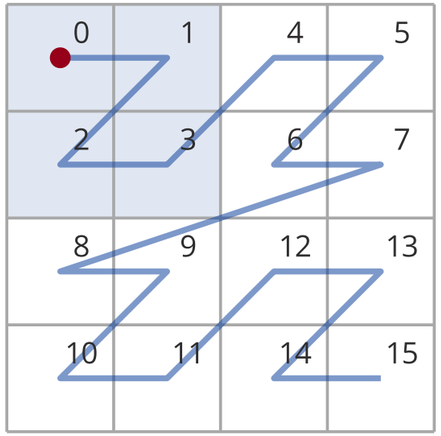


Keeping neighbours close: Z-order

Row-major (the formula)



Z-order (Morton)



Why it matters here: big aggregates and map tiles are stored and fetched by key.

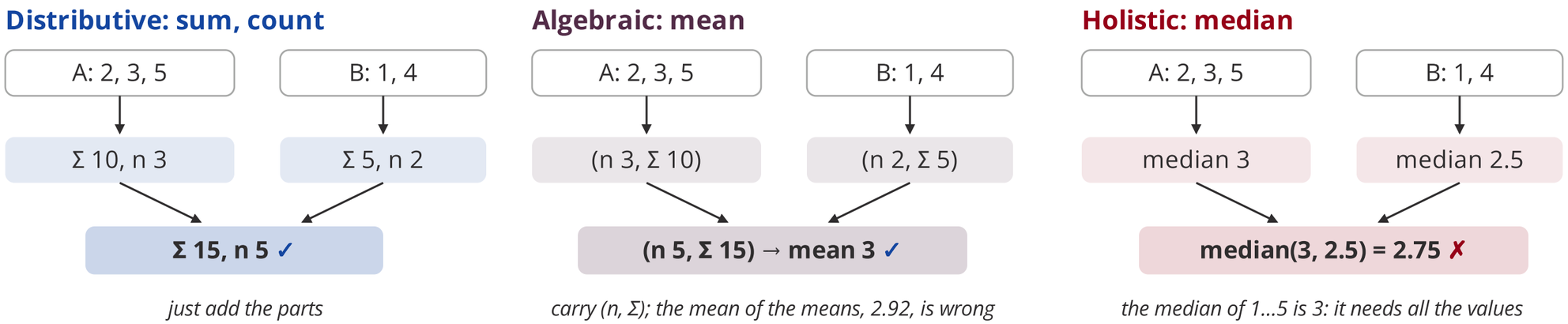
- A zoom or pan window is a rectangle of bins: in Z-order a few long runs of keys, in row-major order one short run per row of bins. Fewer disk reads, cache misses, files touched.
- Map tiles are keyed this way: a tile's key starts with its parent's key (next sections).

- Row-major order keeps only *x*-neighbours adjacent
- Z-order (Morton) interleaves the bits of the bin coordinates: 2D blocks stay together

Microsoft: Bing Maps Tile System · Delta Lake docs: Z-ordering · Moon et al., IEEE TKDE 13(1), 2001

Per-bin summaries: which ones distribute?

	Examples	Combining partial results
Distributive	count, sum, min, max	directly: the sum of the counts
Algebraic	mean, sd, higher moments	via a fixed-size state, e.g. (n, Σx, Σx^2)
Holistic	median, quantiles, mode, count distinct	no bounded state: pay in full, or approximate



Categories: Gray et al., “Data Cube”, Data Mining and Knowledge Discovery 1(1), 1997

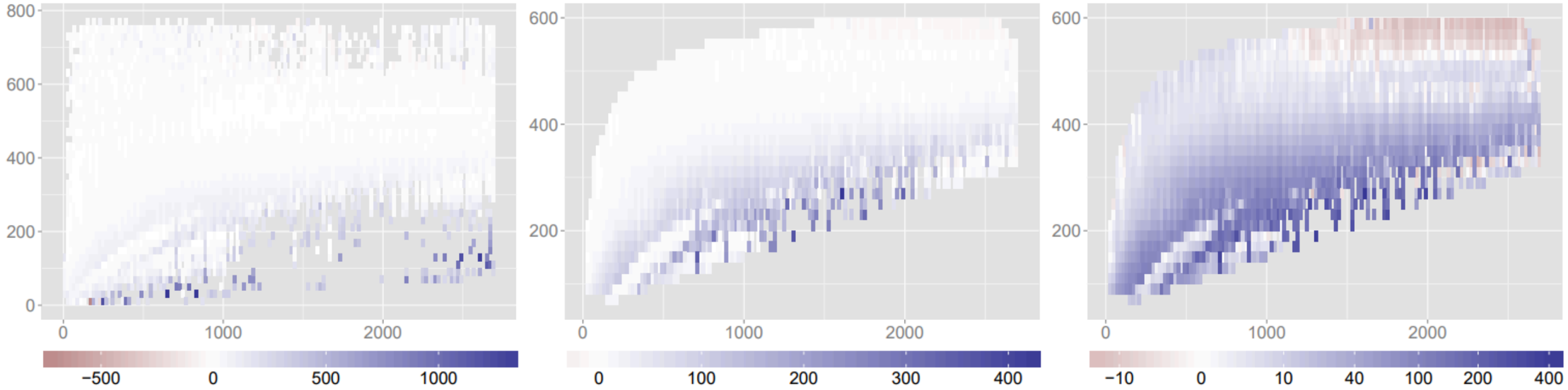
Holistic summaries: approximate them

Summary	Sketch	Idea
count distinct	HyperLogLog	≈ 2 % error for 10^9 distinct values in 1.5 kB
median	remedian	medians of groups of *b*, then medians of those; O(log n) memory
quantiles	t-digest	clusters shrink toward the tails: accurate extreme quantiles; mergeable
quantiles	KLL	provably optimal space for ε-approximate ranks; mergeable

Flajolet et al., AofA 2007 · Rousseeuw & Bassett, JASA 1990 · Dunning & Ertl, arXiv 2019 · Karnin, Lang & Liberty, FOCS 2016

Smooth, peel, transform

- **Smooth:** a weighted average over neighbouring bins (kernel, bandwidth *h*)
- **Peel:** drop the sparsest outer bins, often under 1 % of the data
- **Transform** the summary before colouring: log, modulus, rank



76 million US flights: average delay (colour, minutes) by distance (x, miles) and speed (y, mph). Left: all bins; centre: peeled to the middle 99.5 % of the data; right: delay modulus-transformed. Faster flights have shorter delays; a group of short, slow flights has almost none.

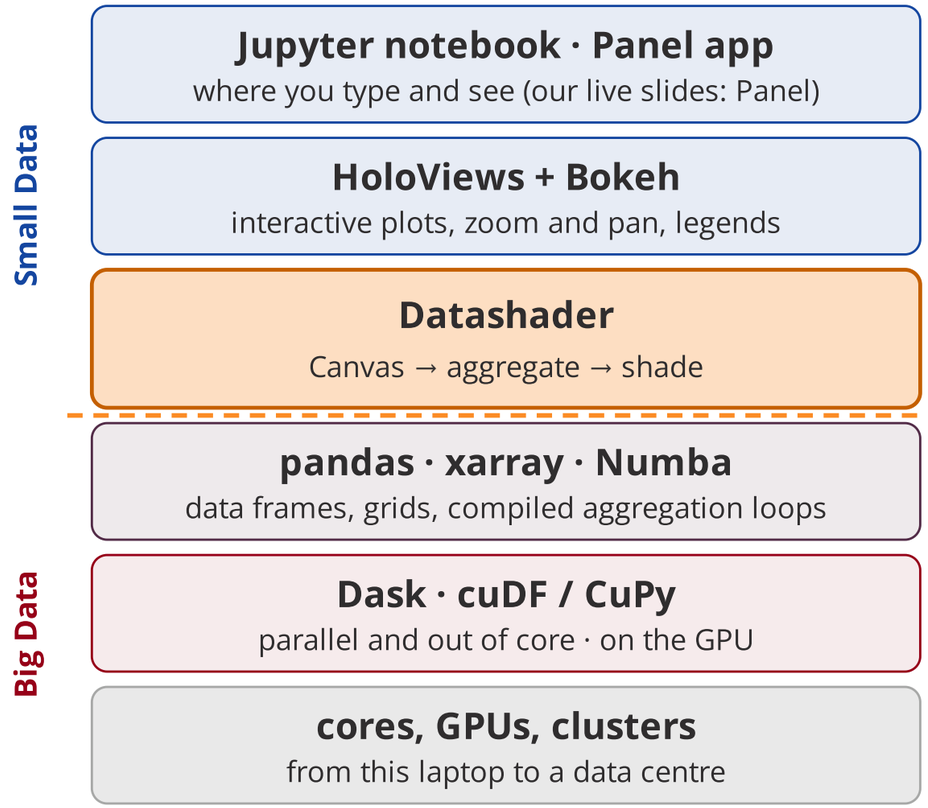
H. Wickham: Bin-summarise-smooth: a framework for visualising large data, tech report, 2013, Fig. 1

ftsrg

Datashader: the framework at pixel resolution

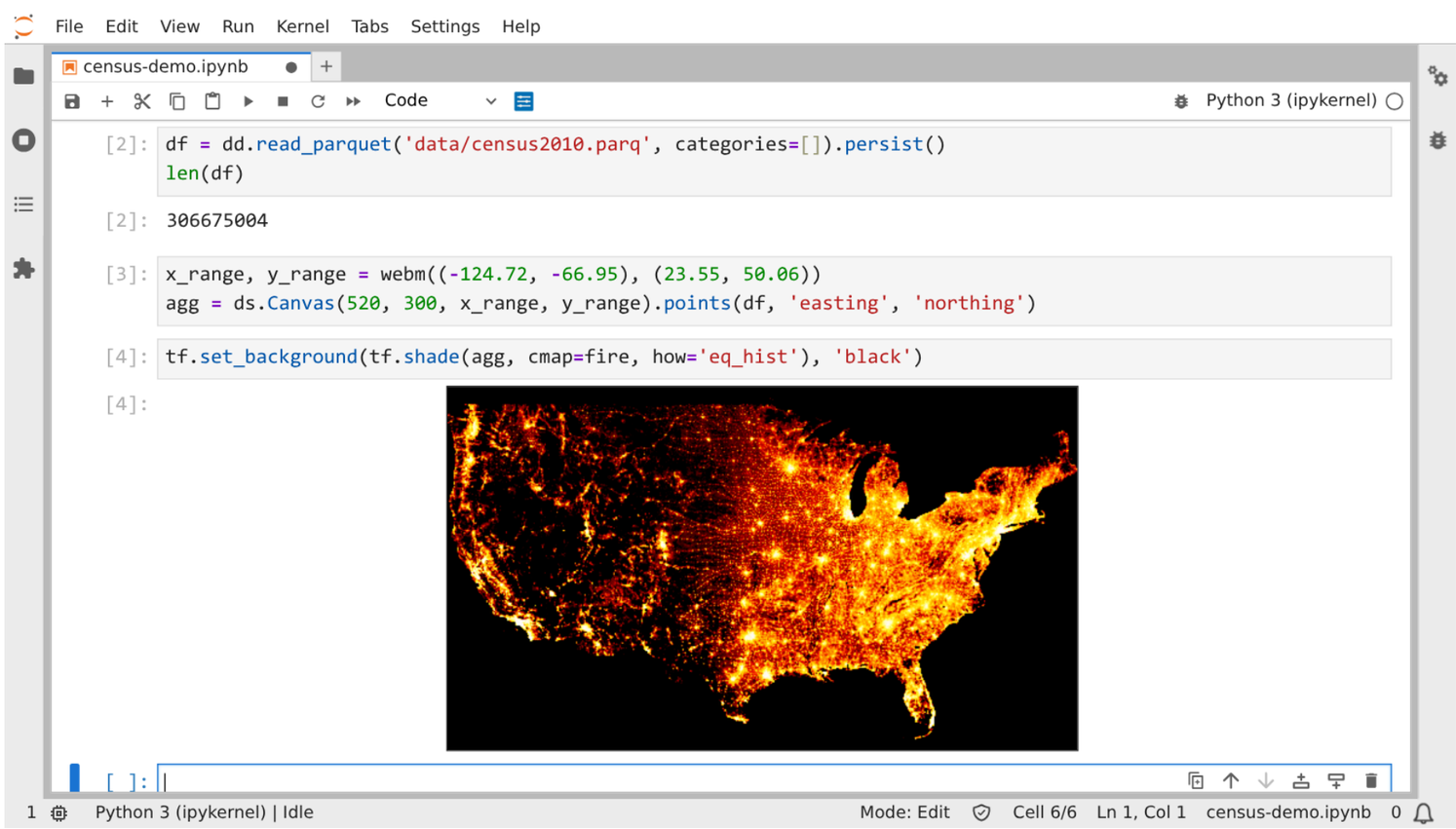
Datashader: a Python library

- Open source (BSD), part of the HoloViz family; developed with Anaconda's support
- Install: `pip install datashader` (or conda); version 0.19.1, May 2026
- Builds on NumPy, pandas, xarray and Numba; Dask for data beyond memory, cuDF / CuPy for GPUs
- datashader.org · github.com/holoviz/datashader



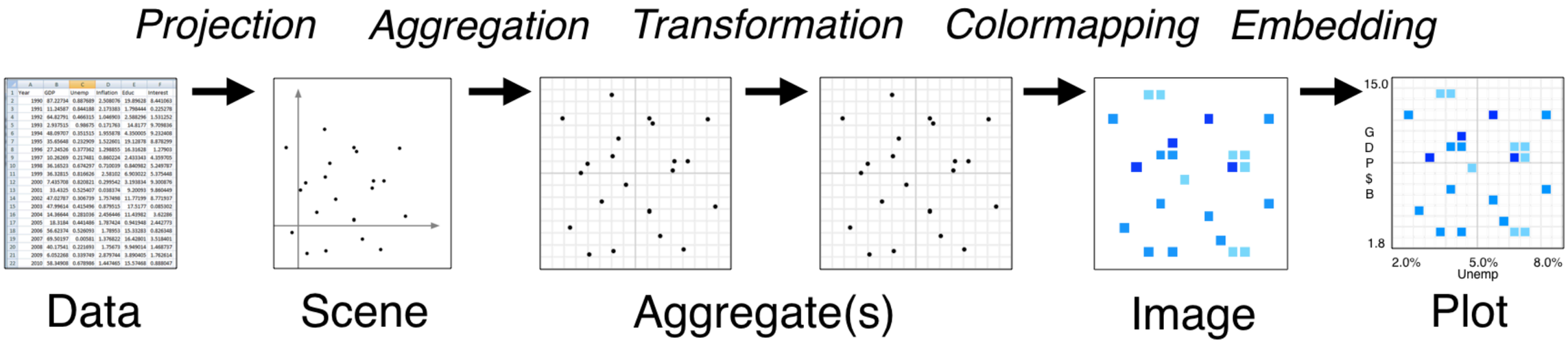
Where datashader usually lives: Jupyter

- A **notebook** mixes text, code cells and their outputs in one document, in the browser
- The code runs in a **kernel** (here: Python) that keeps its variables between cells
- Images and interactive plots appear right under the cell
- The census example is a notebook; our live slides are its cells, lifted out



Project Jupyter · Animation: JupyterLab 4.6 running the case study's first steps on the real data

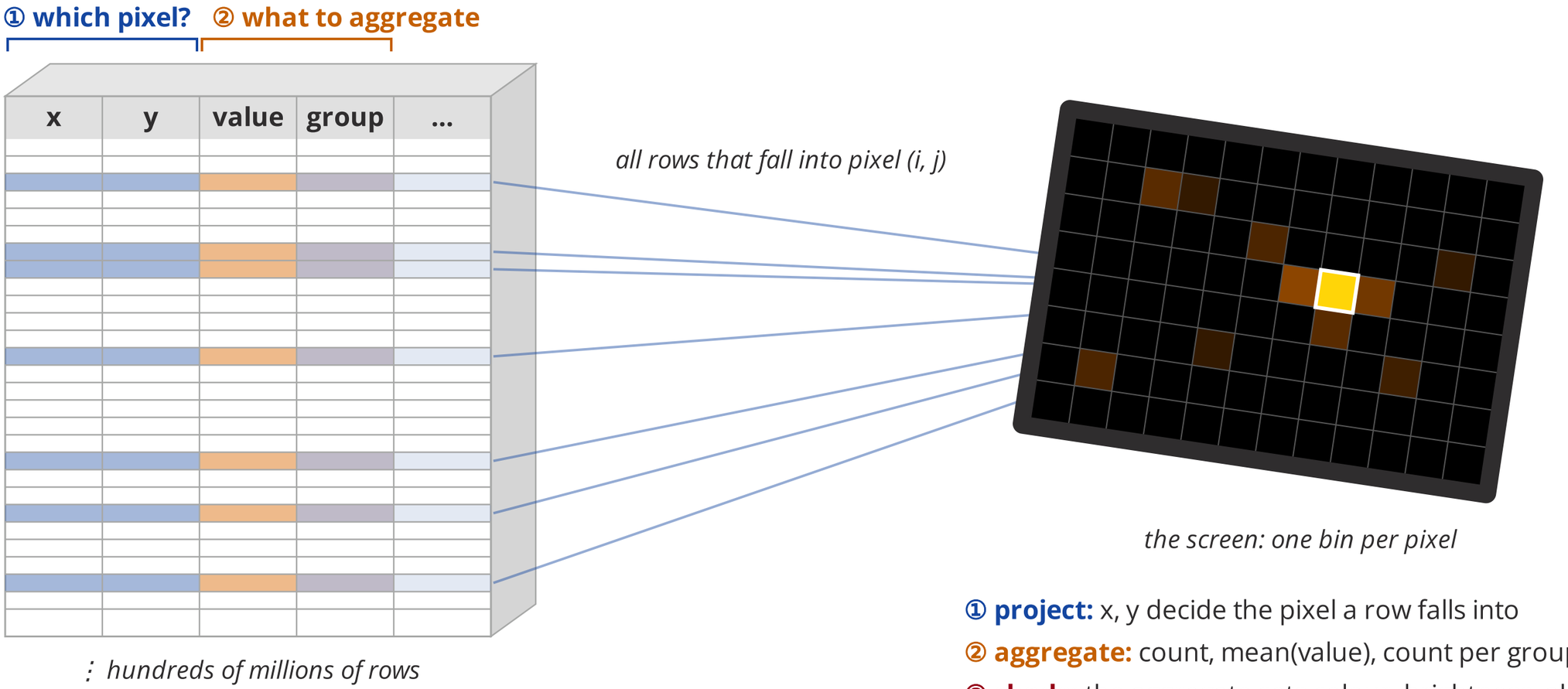
The datashader pipeline



- Five explicit stages; every intermediate result is ordinary data

datashader.org: Pipeline

From rows to one pixel



⋮ *hundreds of millions of rows*

① **project:** x, y decide the pixel a row falls into

② **aggregate:** count, mean(value), count per group

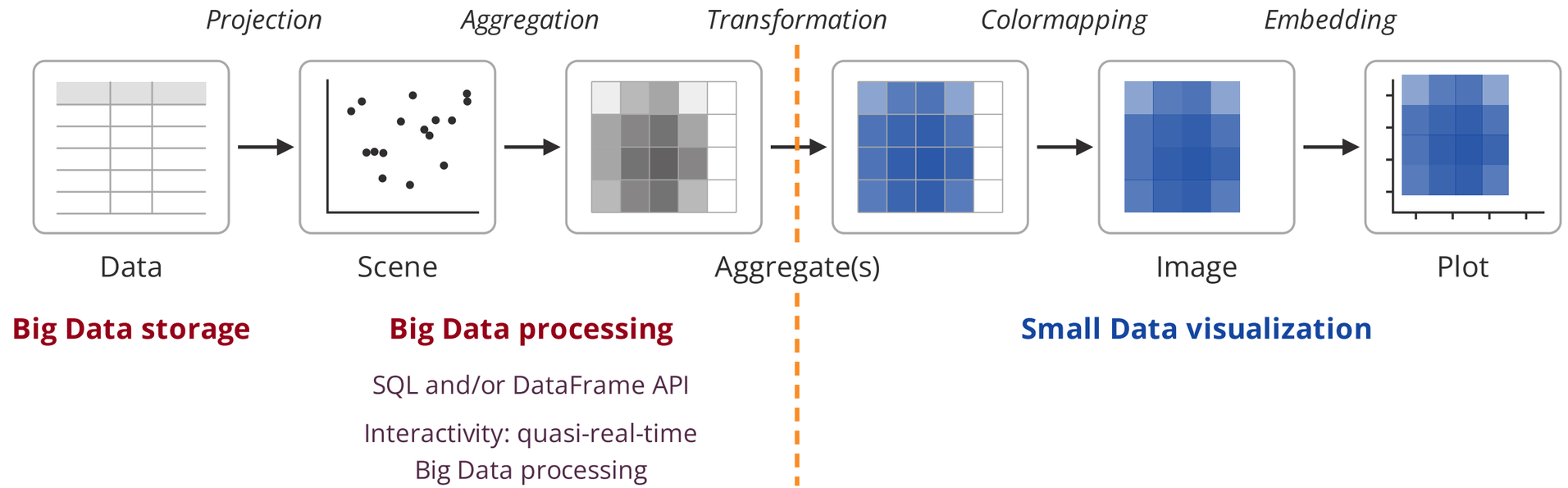
③ **shade:** the aggregate sets colour, brightness, alpha

The same pipeline, read through the framework

Framework	Datashader
Bin	`Canvas(plot_width, plot_height, x_range, y_range)`: one bin per pixel
Summarize	the reduction: `count()`, `mean('col')`, `by('race')`, …
Smooth, peel	no dedicated step; the aggregate is an xarray array, open to any array operation
Transform + colour	`shade(how=…)` with `'linear'`, `'log'`, `'eq_hist'`
Plot	embedding in Bokeh / HoloViews (`datashade`, `rasterize`)

`spread` / `dynspread` enlarge points *for display*: not statistical smoothing

A functional architectural view



Big Data side: if the data is really big, clusters of machines, GPUs, clusters of GPUs, cloud data services

Small Data side: whatever we like, even underpowered devices: from a desktop down to a smart display at a bus stop

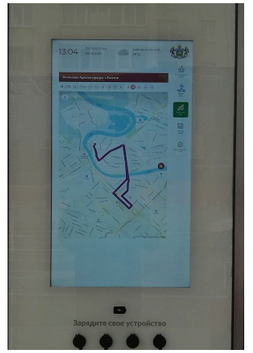


Photo: smart bus stop in Tyumen, RG72, CC BY-SA 4.0, Wikimedia Commons, cropped

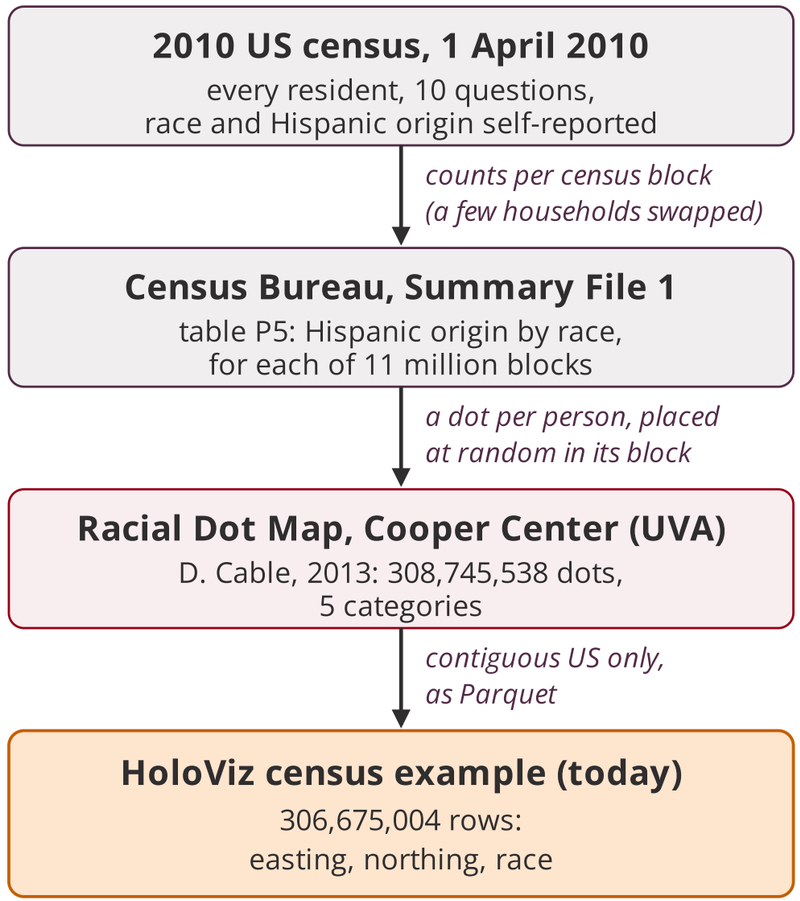
Case study: 300 million people, one pixel grid

The data: the 2010 US census

The census. Every ten years the US counts every resident (Constitution, Art. I §2), to share out the seats of the House of Representatives. Census day 2010: 1 April; 10 questions per household; 308,745,538 people.

Privacy. Answers are confidential by law (Title 13: staff sworn for life, up to 5 years in prison); personal records stay closed for 72 years. What gets published are counts per census block, with some households swapped between nearby blocks (2020: differential privacy).

What we get. No individuals: one dot per counted person, at a random spot inside its block (11 million blocks, 28 people on average). The race column is the census's self-reported category: **w**, **b**, **a** = non-Hispanic White, Black, Asian; **h** = Hispanic, of any race; **o** = everyone else.



census.gov: 2010 Census Briefs C2010BR-01, -02; Title 13 U.S.C. · Cooper Center: Racial Dot Map (2013, archived) · [examples.holoviz.org: Census]

A note on ethics: race in the data

- **Why race is in the US census:** asked by law and self-identified, to enforce civil and voting rights. The Bureau calls its categories “a social definition of race”, not a biological one.
- **Seen from Europe,** asking people for their race is unusual and uncomfortable. The GDPR prohibits processing data “revealing racial or ethnic origin” except in narrow cases, and adds that the term does not imply “acceptance … of theories which attempt to determine the existence of separate human races”. France bars ethnic statistics; Hungary’s 2022 census asked about ethnicity only voluntarily, and not about race at all.
- **Why we still use this data:** it is a famous, public, aggregated benchmark for big-data visualisation. We read the maps as a technical demo: where people were counted, in the census’s own categories. They say nothing about individuals, and nothing for or against any group.
- **The lesson that carries over:** visualisation persuades. The same map can inform or stigmatise; choose what you show, and think about how it will be read.

census.gov: About race · GDPR Art. 9(1) and Recital 51 · Conseil constitutionnel, décision 2007-557 DC · KSH: Census 2022 questionnaire

The data: one row per person

```
1 df = dd.read_parquet('./data/census2010.parq',
      categories=[])
2 df = df.categorize('race')
3 df = df.persist()
4 df.head()
```

	easting	northing	race
0	-12418767.0	3697425.00	h
1	-12418512.0	3697143.50	h
2	-12418245.0	3697584.50	h
3	-12417703.0	3697636.75	w
4	-12418120.0	3697129.25	h

▶ Run (Shift+Enter) Wall time: 28.44 s

Try `len(df)`, then `df.race.value_counts().compute()`: each is a pass over all 300 M rows

Aggregation: 300 million points into one grid

```
1 cvs = ds.Canvas(plot_width, plot_height, *webm(*USA
      ))
2 agg = cvs.points(df, 'easting', 'northing')
3 agg
```

xarray.DataArray (**northing**: 525, **easting**: 900)

```
array([[0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]], shape=(525, 900), dtype=uint32)
```

▼ Coordinates:

northing	(northing)	float64	2.702e+06 2.709e+06 ... 6.453e+06
easting	(easting)	float64	-1.388e+07 ... -7.456e+06

▼ Attributes:

x_range :	(np.float64(-13883766.89173708), np.float64(-7452839.908609666))
y_range :	(np.float64(2698668.7393027707), np.float64(6456673.275328229))

▶ Run (Shift+Enter) Wall time: 0.53 s

Try `agg.sum()`: how many of the 306 M people fall inside this frame?

Linear colour scale: almost nothing

```
1 export(tf.shade(agg, cmap=cm(Greys9), how='linear'))
```



▶ Run (Shift+Enter) Wall time: 0.03 s

Try `how='cbrt'` (cube root): between linear and log

People per pixel, linear

0 maximum

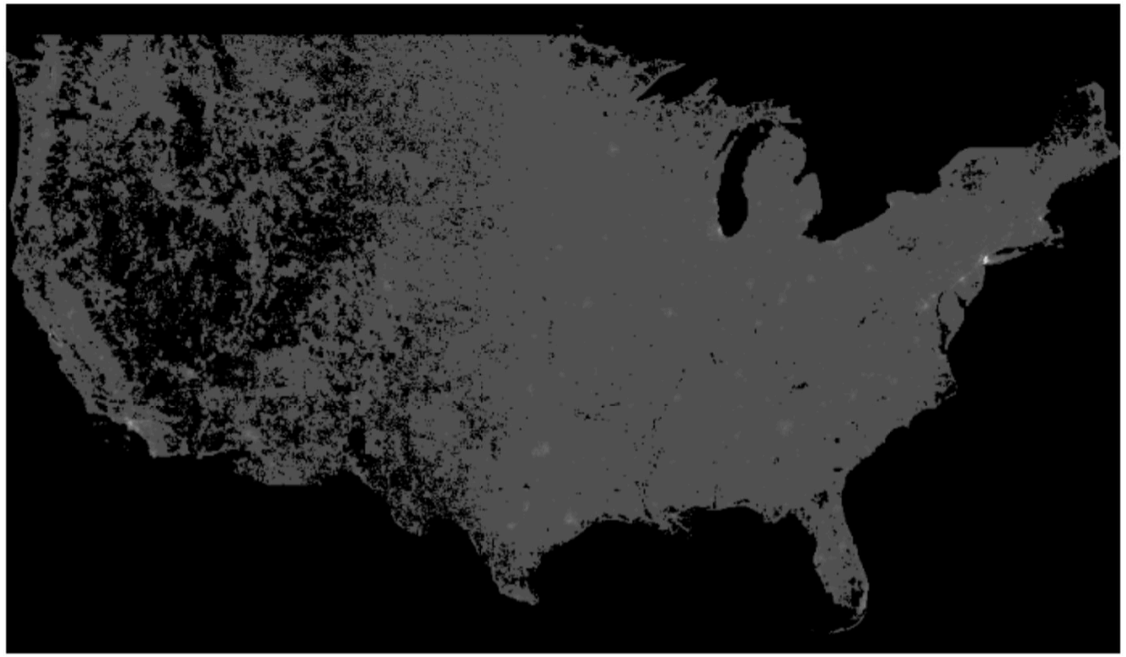
ftsrg

Undersampling: lift the floor

```
export(tf.shade(agg, cmap=cm(Greys9, 0.25), how
    ='linear'))
```

▶ Run (Shift+Enter)

Wall time: 0.03 s



Try other floors: `cm(Greys9, 0.5)`, or `0` to get the previous slide back

People per pixel, linear; ■ black nobody

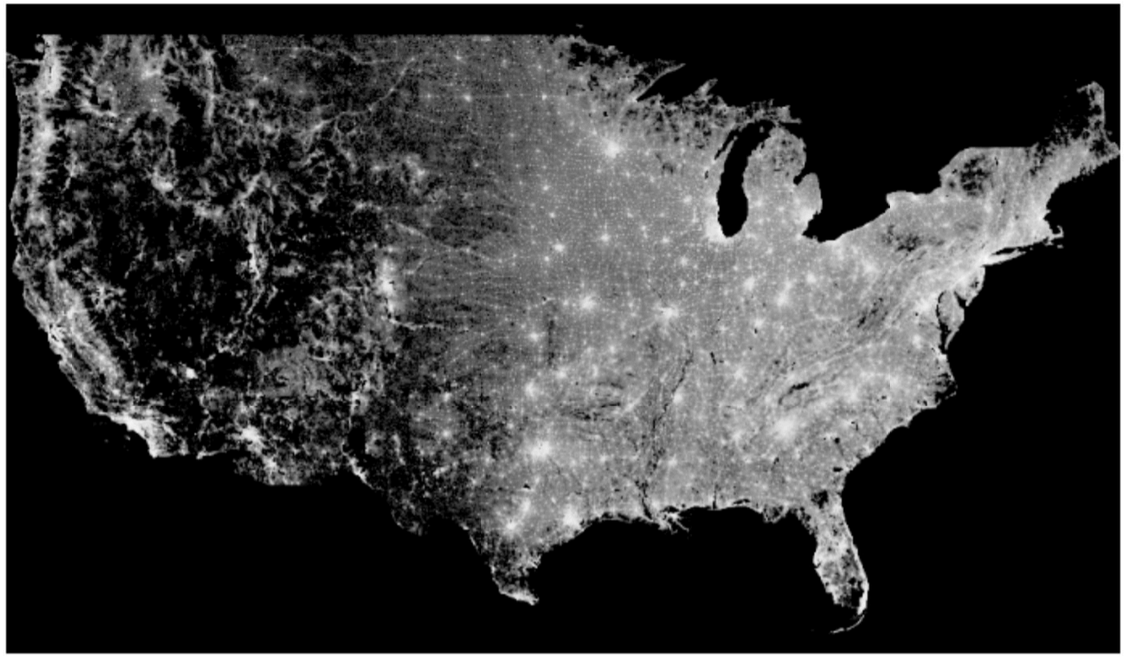
1 maximum

Logarithmic scale: suddenly, structure

```
export(tf.shade(agg, cmap=cm(Greys9, 0.2), how='log'
    ))
```

▶ Run (Shift+Enter)

Wall time: 0.03 s



Try `how='cbrt'`: another fixed transform, another guess

People per pixel, logarithmic; ■ black nobody

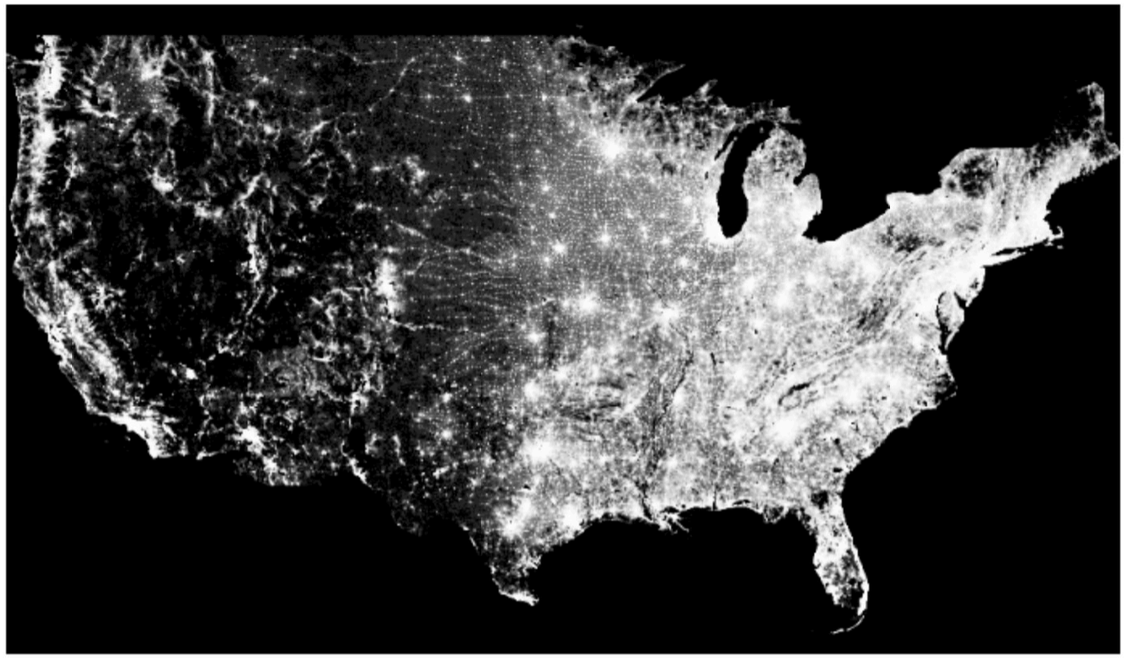
1 maximum

ftsrg

Histogram equalization: no parameter to guess

```
export(tf.shade(agg, cmap=cm(Greys9, 0.2), how
    ='eq_hist'))
```

▶ Run (Shift+Enter) Wall time: 0.05 s



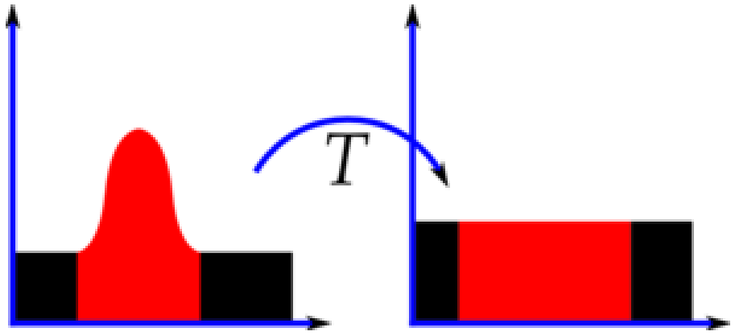
Try `cm(Greys9)` without the floor; then `how='linear'` again, to compare

People per pixel, histogram-equalized (by rank); ■ black nobody

fewest — most

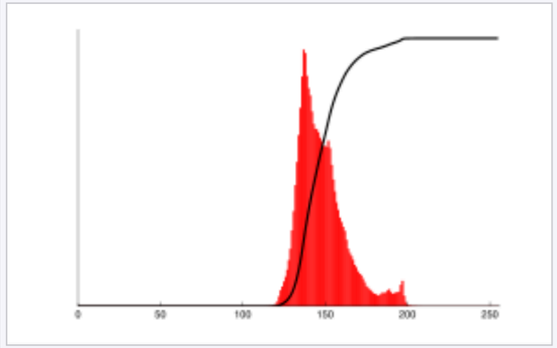
Histogram equalization

Goal: use every colour for about the same number of pixels





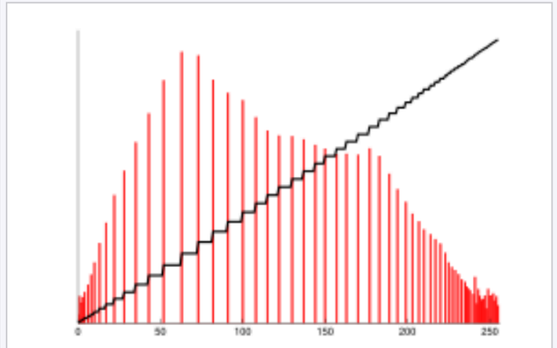
Before Histogram Equalization



Corresponding histogram (red) and cumulative histogram (black)



After Histogram Equalization



Corresponding histogram (red) and cumulative histogram (black)

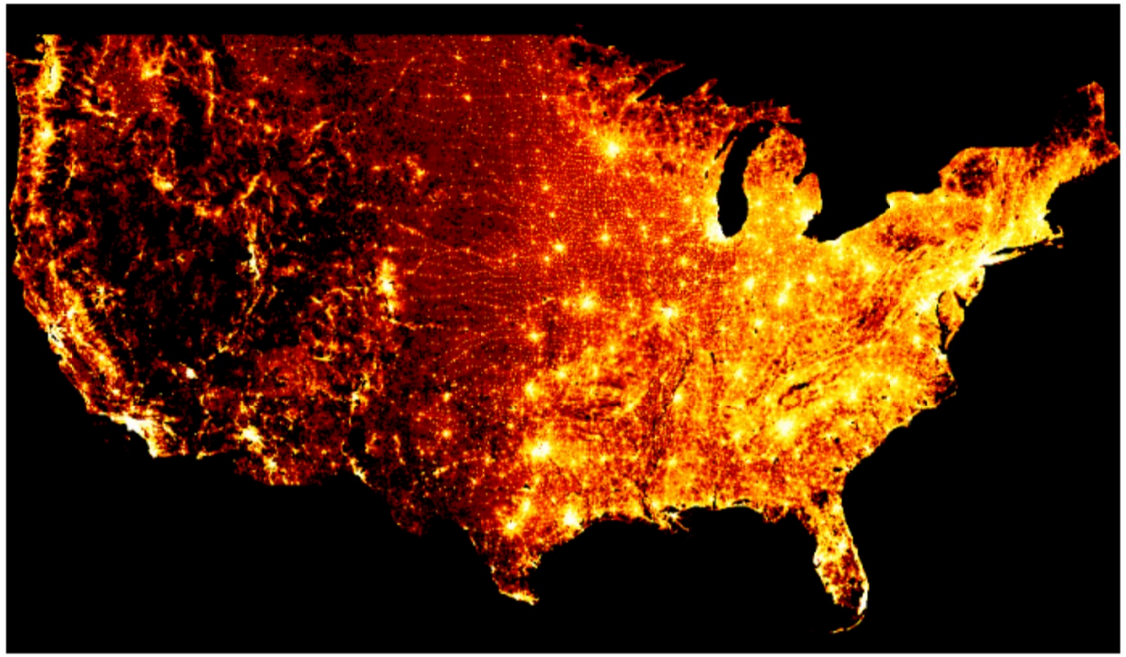
Wikipedia: Histogram equalization

A perceptually uniform colormap: fire

```
from colorcet import fire
export(tf.shade(agg, cmap=cm(fire, 0.2), how
    ='eq_hist'))
```

▶ Run (Shift+Enter)

Wall time: 0.07 s



Try `how='linear'`: does a better colormap rescue a linear scale?

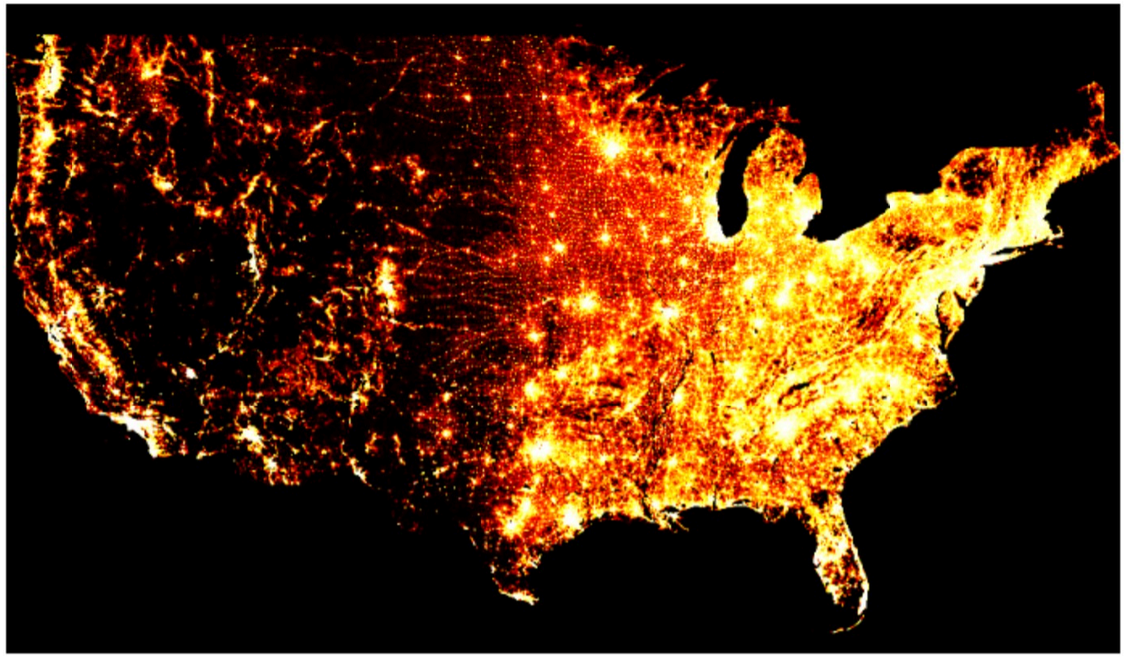
People per pixel, histogram-equalized; ■ black nobody

fewest most

…versus matplotlib's hot

```
1 from matplotlib.cm import hot
2 export(tf.shade(agg, cmap=hot, how='eq_hist'))
```

▶ Run (Shift+Enter) Wall time: 0.05 s



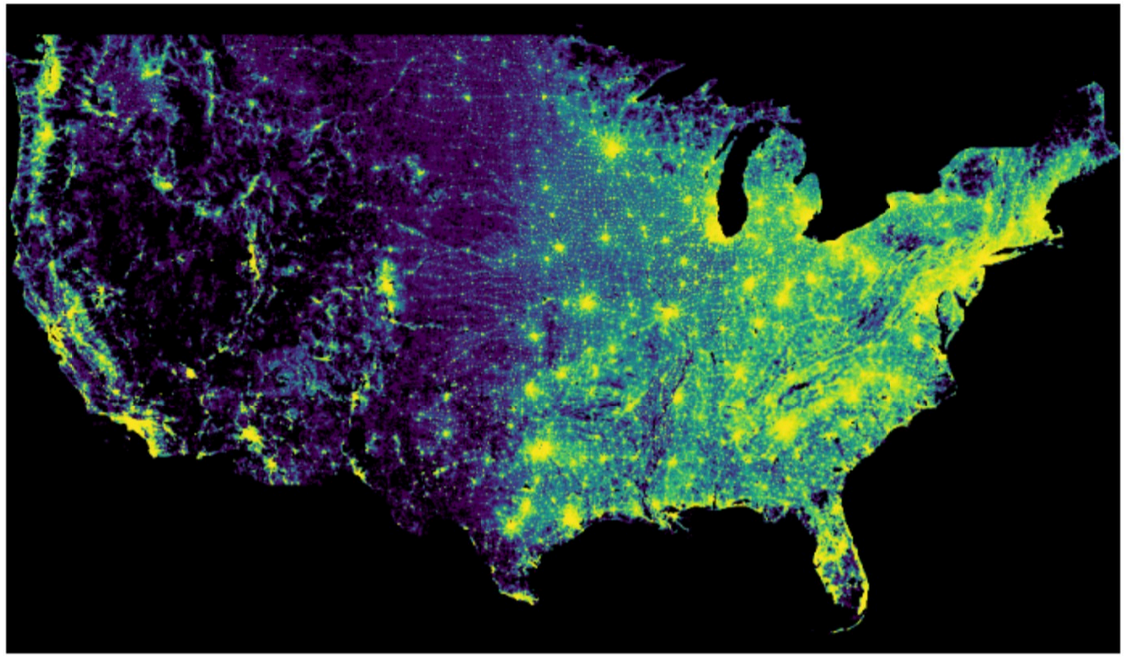
Flip back and forth with the previous slide: where does hot hide detail?

People per pixel, histogram-equalized

fewest most

viridis

```
1 from datashader.colors import viridis
2 export(tf.shade(agg, cmap=cm(viridis), how='eq_hist'
      ))
```



▶ Run (Shift+Enter) Wall time: 0.07 s

Try another colorcet map: `from colorcet import bmy`, then `cmap=cm(bmy)`

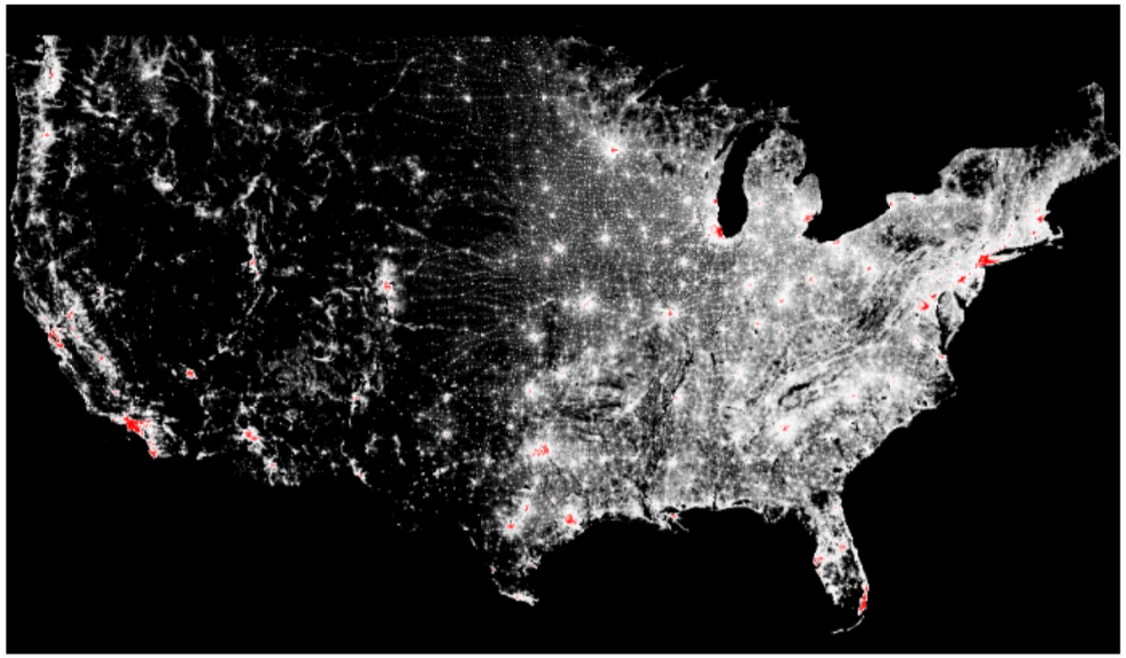
People per pixel, histogram-equalized

fewest most

A colormap that asks a question: the top 1 %

```
1 grays2 = cm([(i,i,i) for i in np.linspace(0, 255, 99
      )])
2 grays2 += ["red"]
3 export(tf.shade(agg, cmap=grays2, how='eq_hist'))
```

▶ Run (Shift+Enter) Wall time: 0.06 s



Try the top 10 %: `linspace(0, 255, 90)` and `grays2 += ["red"]*10`

People per pixel, histogram-equalized

fewest most

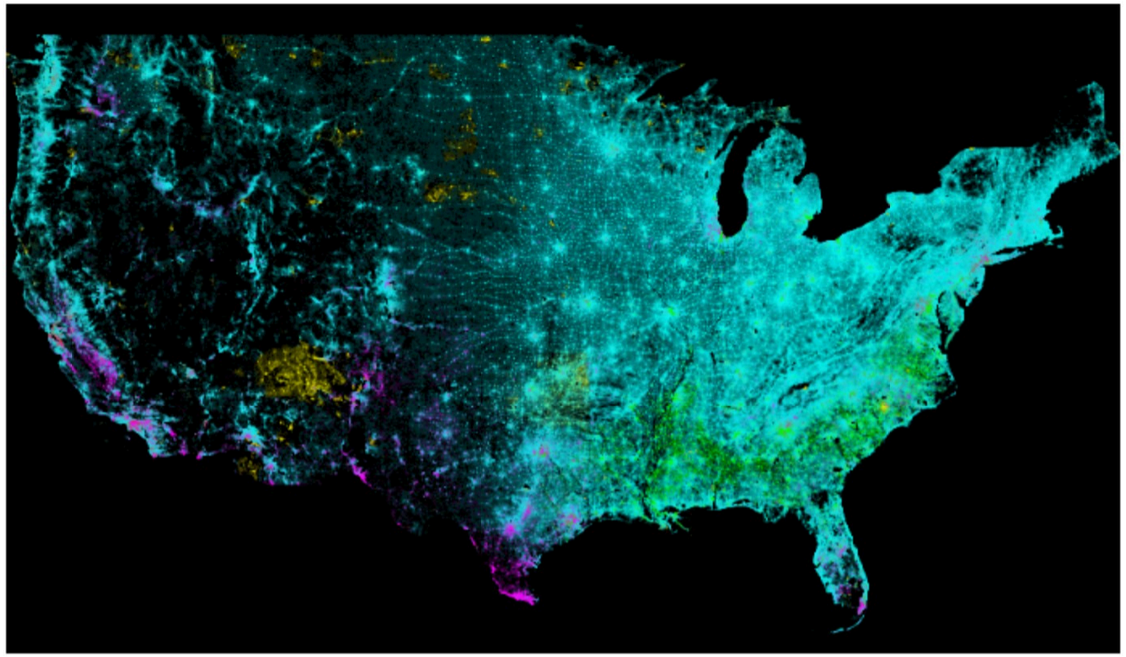
top 1 % of populated pixels

Categories: one grid per race

```
def create_image(longitude_range, latitude_range,
                 w=plot_width, h=plot_height):
    x_range, y_range = webm(longitude_range,
        latitude_range)
    cvs = ds.Canvas(plot_width=w, plot_height=h,
                    x_range=x_range, y_range=y_range
                        )
    agg = cvs.points(df, 'easting', 'northing', ds
        .by('race'))
    return tf.shade(agg, color_key=color_key, how
        ='eq_hist')

export(create_image(*USA))
```

▶ Run (Shift+Enter) Wall time: 1.27 s

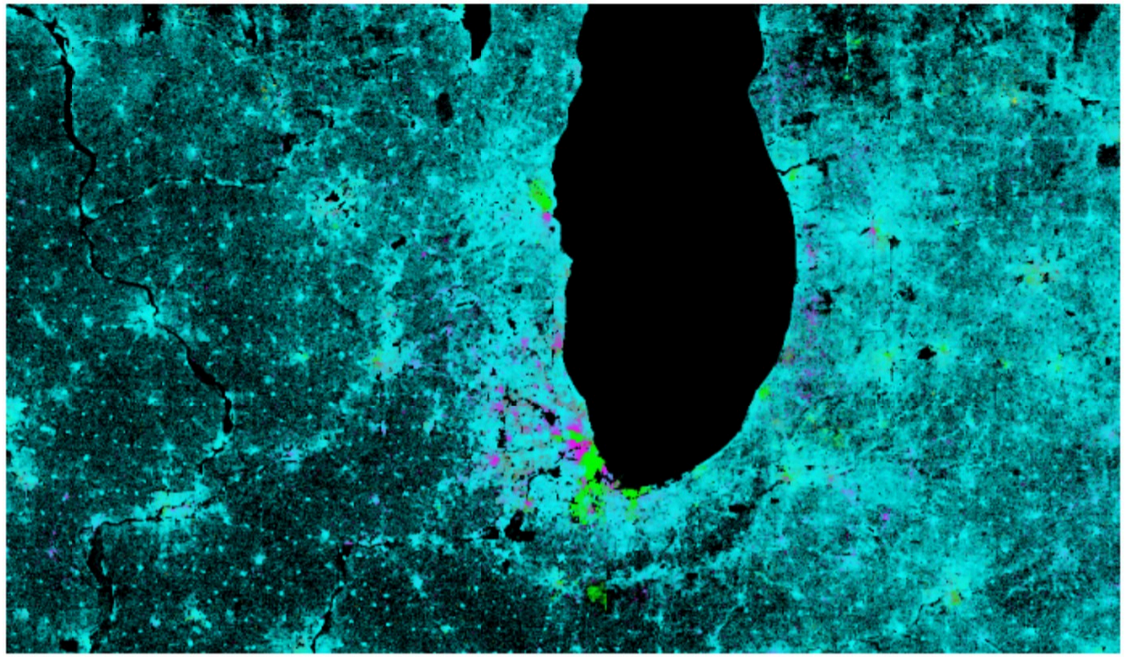


Try another region: `export(create_image(*cities['Atlanta']))`

White Black Asian Hispanic Other
Hue: mix of the groups in a pixel; brightness: number of people (equalized)

Lake Michigan

```
1 export(create_image(*LakeMichigan))
```



▶ Run (Shift+Enter)

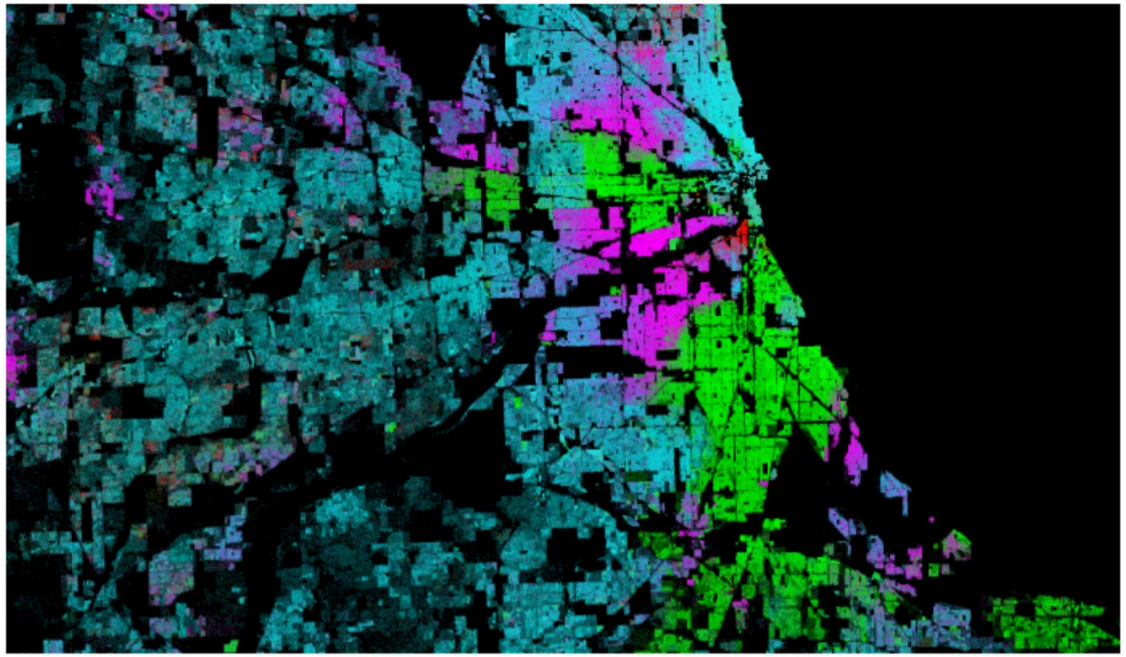
Wall time: 0.51 s

Try your own box: `export(create_image((-88.0, -87.5), (41.7, 42.1)))`

White Black Asian Hispanic Other
Hue: mix of the groups in a pixel; brightness: number of people (equalized)

Chicago

```
1 export(create_image(*Chicago))
```



▶ Run (Shift+Enter)

Wall time: 0.50 s

Try fewer, bigger bins: `create_image(*Chicago, w=225, h=131)`

White Black Asian Hispanic Other

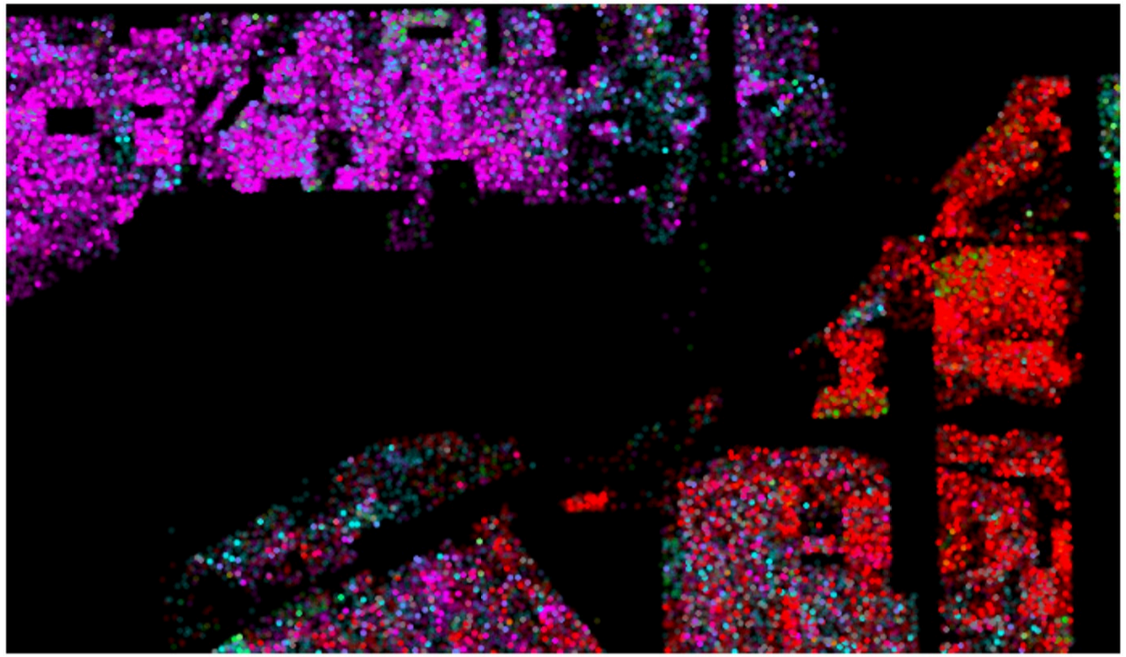
Hue: mix of the groups in a pixel; brightness: number of people (equalized)

Chinatown: individual people

```
export(tf.spread(create_image(*Chinatown), px
    =plot_width//400))
```

▶ Run (Shift+Enter)

Wall time: 0.39 s



Try `px=0` (no spreading), then `px=5`: spreading changes the display, not the data

White Black Asian Hispanic Other
Each dot: one person

Other cities

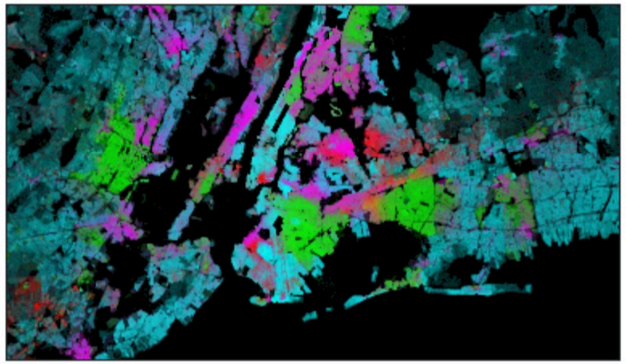
White Black Asian Hispanic Other
Hue: mix of the groups; brightness: number of people

```
1 tf.Images(*[export(create_image(*r, w=380, h=220)).rename(name)
2             for name, r in cities.items()]).cols(3)
```

▶ Run (Shift+Enter) Wall time: 1.32 s

Try `w=95, h=55`: 16× fewer bins; does the wall time drop?

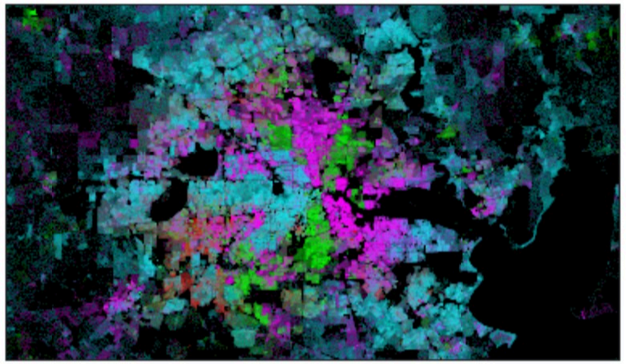
New York



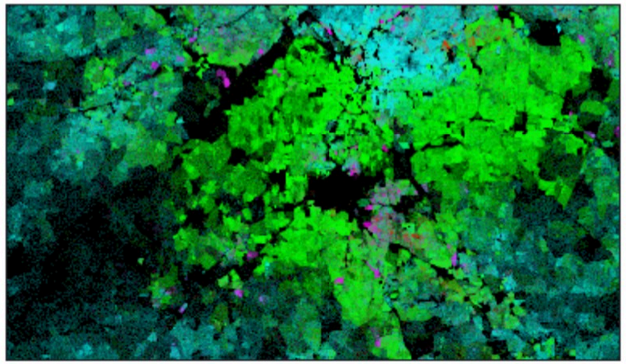
Los Angeles



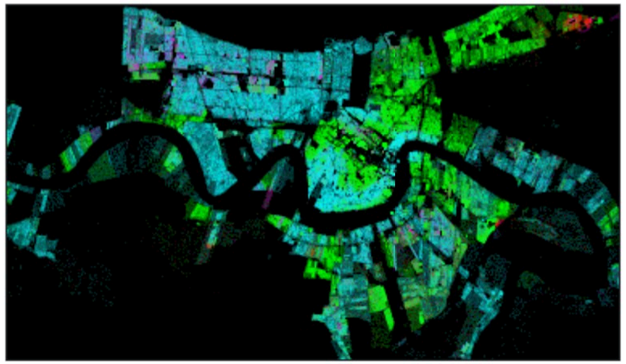
Houston



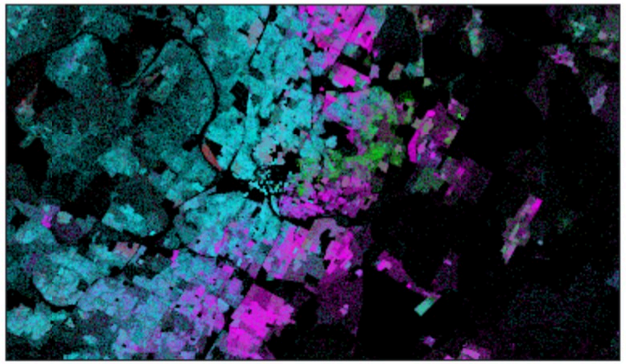
Atlanta



New Orleans



Austin

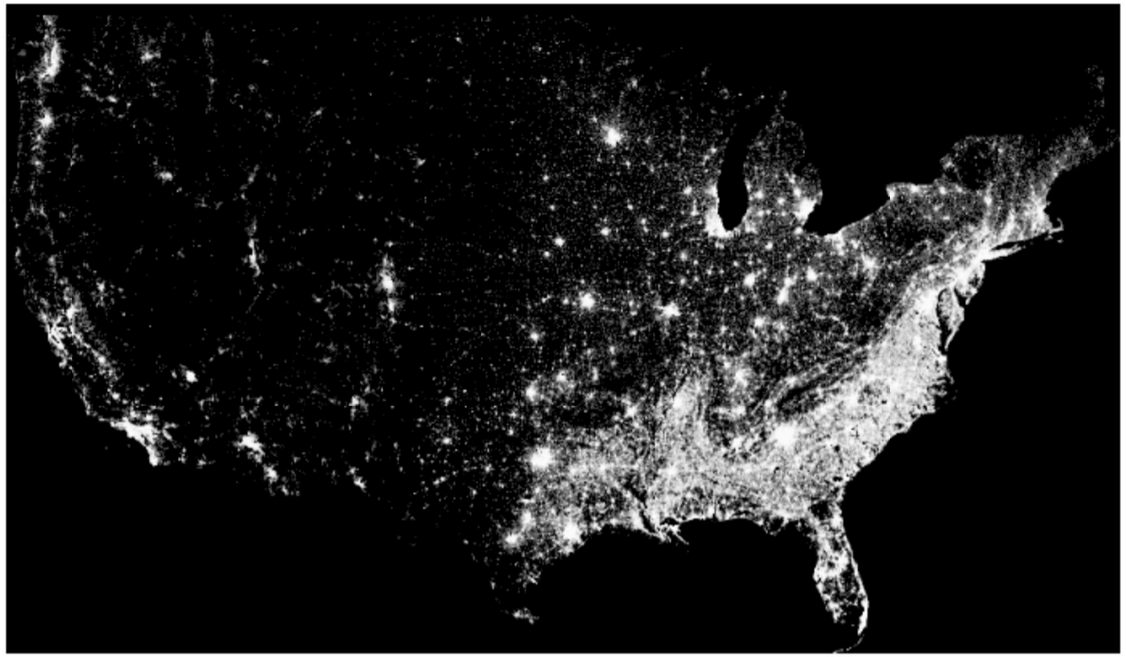


Aggregates are data: querying the grid

```
1 cvs = ds.Canvas(plot_width=plot_width, plot_height
      =plot_height)
2 aggc = cvs.points(df, 'easting', 'northing', ds.by
      ('race'))
3 export(tf.shade(aggc.sel(race='b'), cmap=cm(Greys9,
      0.25),
4                 how='eq_hist'))
```

▶ Run (Shift+Enter)

Wall time: 1.00 s



Try `race='h'` (Hispanic) or `race='a'` (Asian)

Black residents per pixel, histogram-equalized; ■ black none

fewest most

ftsrg

Pixels where four groups all live

```
1 agg2 = aggc.where((aggc.sel(race=['w','b','a','h'])
      > 0)
2                   .all(dim='race')).fillna(0)
3 export(tf.shade(agg2, color_key=color_key, how
      ='eq_hist'))
```



Try requiring just two groups: `['w','b']`

▶ Run (Shift+Enter)

Wall time: 0.14 s

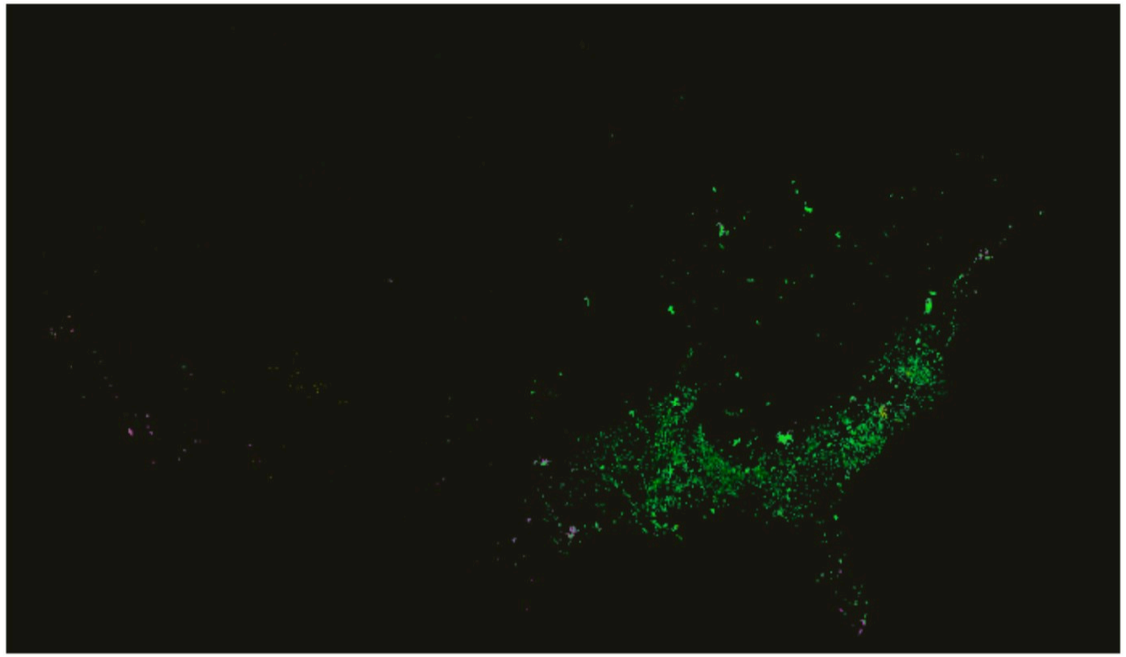
White Black Asian Hispanic Other

Only pixels where White, Black, Asian and Hispanic residents all live

ftsrg

More Black than white residents

```
export(tf.shade(aggc.where(aggc.sel(race='w') < aggc
    .sel(race='b'))
                    .fillna(0),
                color_key=color_key, how='eq_hist'))
```



Try `'h'` instead of `'b'`: more Hispanic than white residents

▶ Run (Shift+Enter) Wall time: 0.14 s

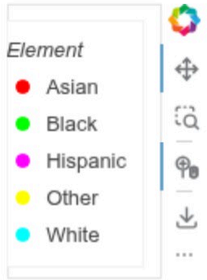
White Black Asian Hispanic Other
Only pixels with more Black than white residents



Live: datashader behind an interactive plot

A colour key needs a plotting library

```
races = {'w':'White', 'b':'Black', 'a':'Asian', 'h':'Hispanic', 'o':'Other'}
color_points = hv.NdOverlay({races[k]: hv.Points([webm(-80,40)]).opts(color=v, size=0)
                             for k, v in color_key.items()})
color_points.opts(clone=True, xaxis=None, yaxis=None, height=190, width=135)
```



▶ Run (Shift+Enter)

Wall time: 0.02 s

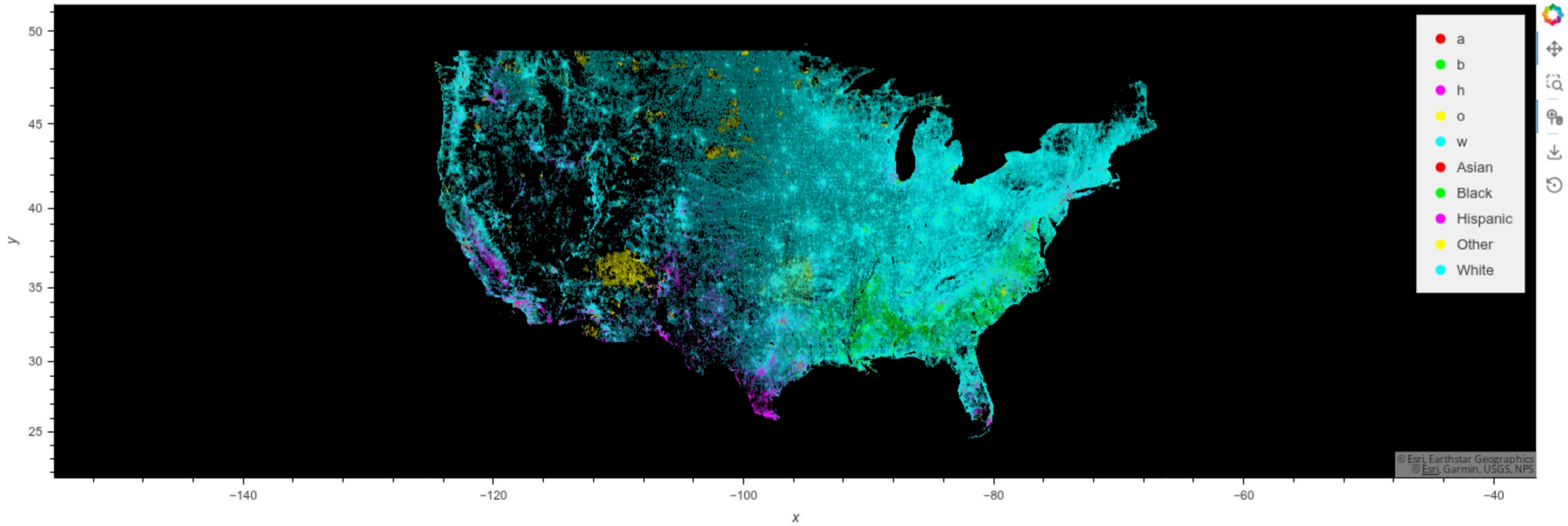
The key is made of five invisible points (`size=0`): try `size=10`

Re-aggregating on every zoom

```
1 points = hv.Points(hv.Dataset(df, kdims=['easting', 'northing'], vdims=['race']))
2 shaded = datashade(points, color_key=color_key, min_alpha=100, aggregator=ds.by('race'))
3 race = dynspread(shaded, threshold=0.8).opts(bgcolor='black')
4 tiles.EsriImagery().opts(alpha=0.5, bgcolor='black') * race * color_points * labels
```

▶ Run (Shift+Enter) Wall time: 0.01 s

Zoom into a city with the mouse wheel: every zoom re-aggregates all 300 M points

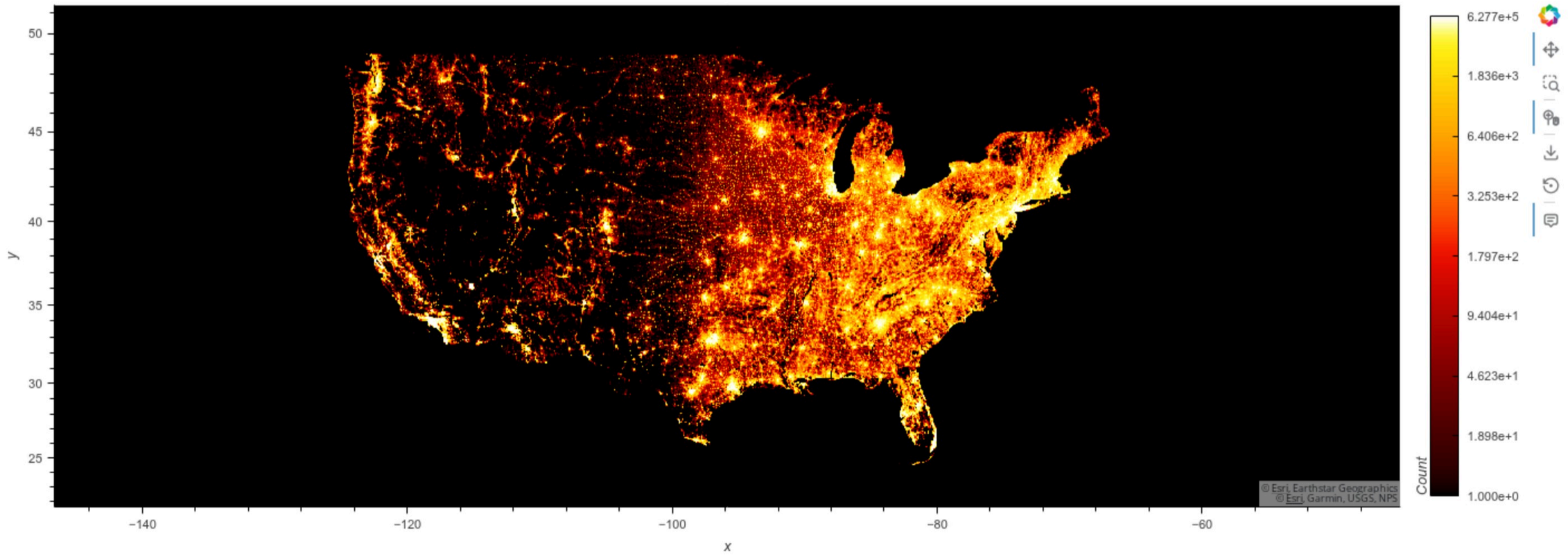


Population density, with hover

```
1 population = dynspread(rasterize(points)).opts(
2     cnorm='eq_hist', cmap='fire', colorbar=True, tools=['hover'])
3 tiles.EsriImagery().opts(alpha=0.5, bgcolor='black') * population * labels
```

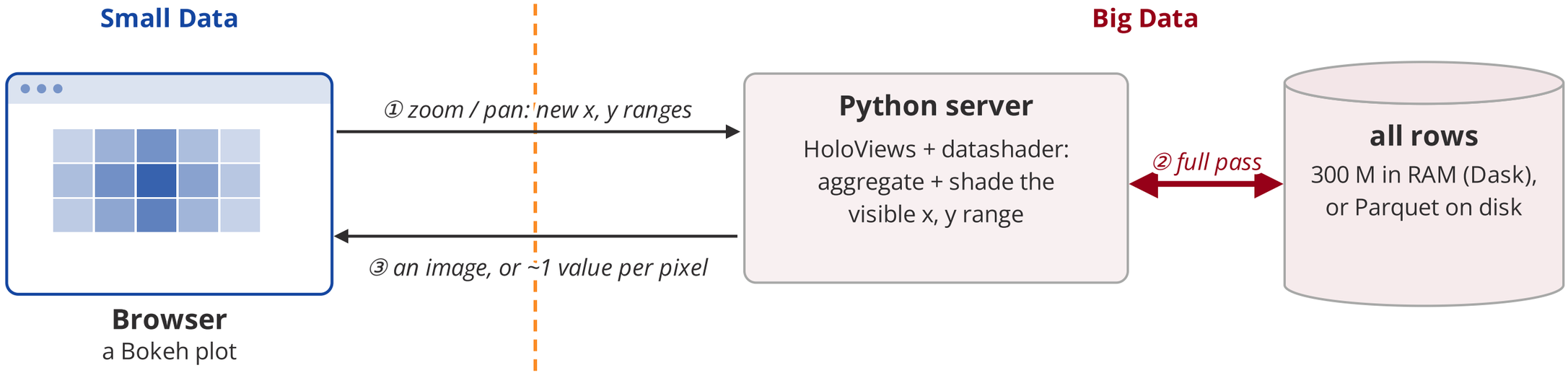
▶ Run (Shift+Enter) Wall time: 0.00 s

Hover over a city: the tooltip shows the count; then try `cnorm='linear'`



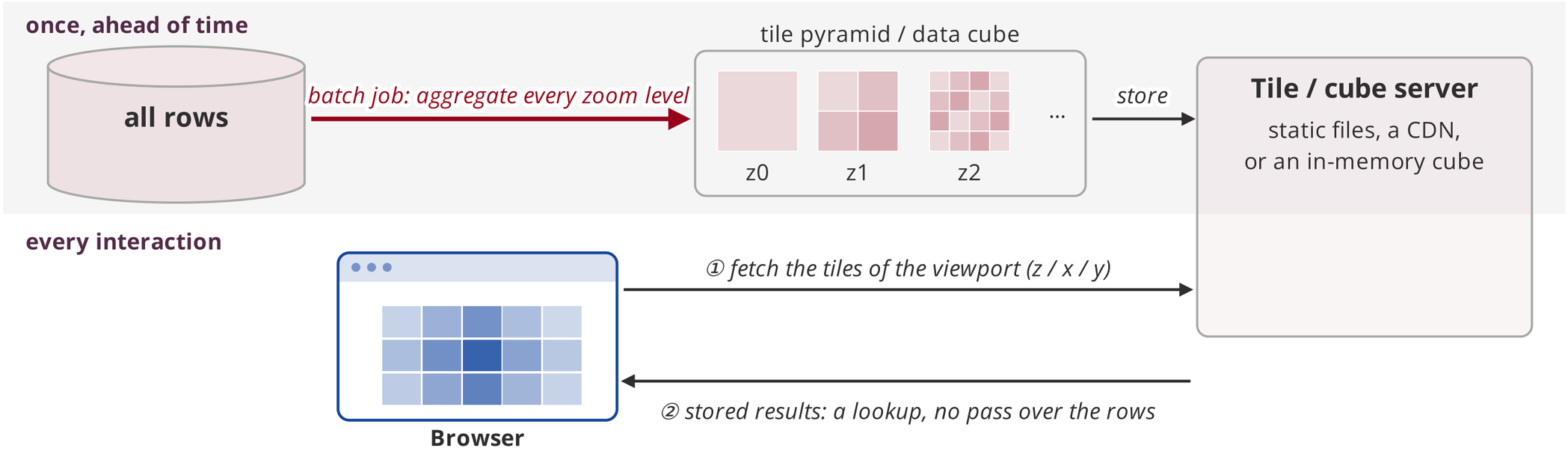
Interactivity at scale: where does aggregation run?

Re-aggregate on demand, on the server



- What the census demo does: every zoom or pan re-runs datashader in Python
- One full pass over the data per interaction; only the image travels
- Any question you can code, always on current data

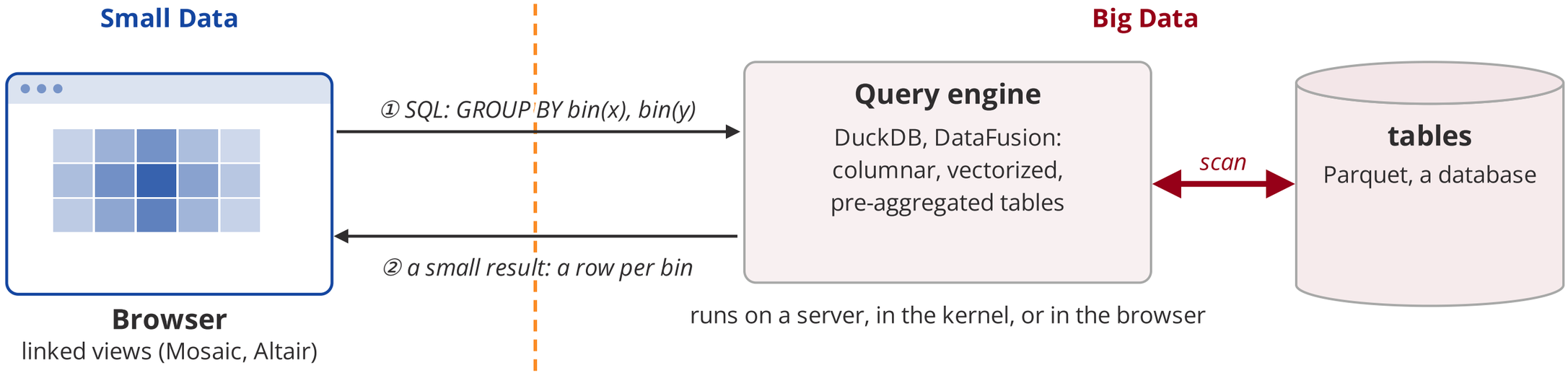
Precompute: tiles and data cubes



- Compute once, serve many: each interaction is a lookup
- Fast, but limited to the choices made in advance

Racial Dot Map (Cooper Center, UVA, 2013; now offline): prerendered tiles, one dot per person · **Nanocubes** (Lins, Klosowski, Scheidegger 2013): in-memory spatiotemporal data cube · **imMens** (Liu, Jiang, Heer 2013): binned “data tiles”, brushed on the GPU

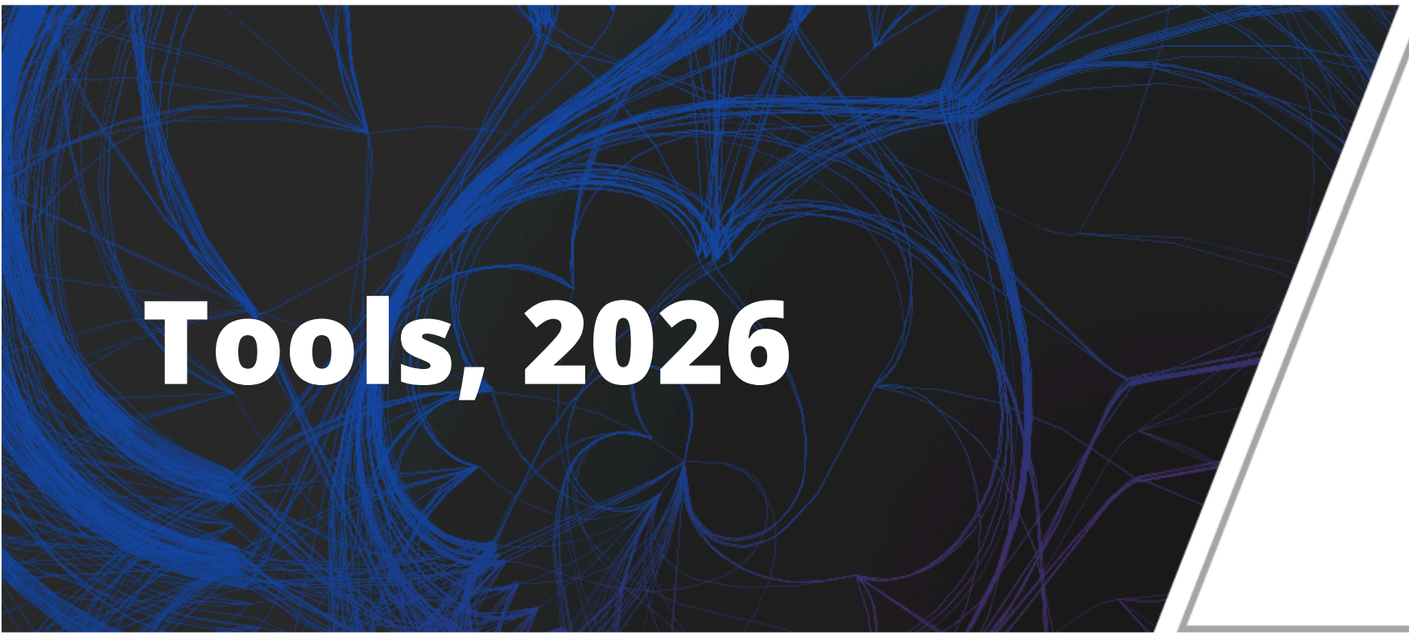
Push aggregation into a query engine



- Each view sends SQL; the engine aggregates; the browser gets a small result
- **Mosaic** (Heer & Moritz, 2023): views query DuckDB; linked views get pre-aggregated tables at pixel resolution
- **VegaFusion** (Kruchten, Mease, Moritz, 2022): Vega and Altair transforms run in a Rust query engine

Three architectures compared

	Latency per interaction	Data freshness	Query flexibility	Cost / infrastructure
Re-aggregate on demand	one pass over the data	always current	anything you can code	a live server holding the data
Precomputed tiles / cubes	a lookup	stale until rebuilt	only what was precomputed	heavy preprocessing and storage, cheap serving
Query engine	one query; engine optimisations help	as current as the database	what SQL can express	an engine: server, kernel or browser



Tools, 2026

Where the computation happens

Browser	Python server / kernel	Query engine	GPU
Plotly interactive charts, WebGL traces	**HoloViz** Datashader, HoloViews, hvPlot, Panel	**DuckDB** in-process SQL; also in the browser (WASM)	**RAPIDS cuDF** GPU DataFrames; back-end for Datashader, Polars
Altair / Vega-Lite declarative grammar, run by Vega in the browser	**Dask** parallel, out-of-core DataFrames and arrays	**Polars** lazy, multithreaded Rust DataFrames	
deck.gl + lonboard GPU map layers (WebGL, WebGPU); lonboard: Jupyter		**Mosaic** linked views whose queries run in DuckDB	
		VegaFusion Vega / Altair transforms on Apache DataFusion	

HoloViz: a cooperating collection

Quick interactive plots

Rasterization for Big Data



Annotate data for visualization
(“easy interactive datashader”)

Data-driven dashboards

holoviz.org



Pitfalls and take-aways

Pitfalls, part 1: too many marks

Pitfall	What goes wrong	Where you saw it
Overplotting	later marks hide earlier ones; draw order decides what you see	one-mark-per-row charts; datashader aggregates instead
Oversaturation	with transparency, every dense area reaches full colour	avoided: the census maps colour counts, not blended marks
Undersampling	a random subsample loses the shape of the distribution	the sampling slides; the census used all 300 M rows

datashader.org: Common plotting pitfalls that get worse with large data

Pitfalls, part 2: too few colours

Pitfall	What goes wrong	Where you saw it
Undersaturation	sparse but non-empty bins look like the background	“lift the floor”: `cm(Greys9, 0.25)`
Underutilized range	nearly all pixels at the two ends of the colormap	“Linear colour scale: almost nothing”; fixed by log, `eq_hist`
Nonuniform colormapping	stretches of equal-looking colours hide detail	colorcet’s fire vs matplotlib’s hot

Always look at both the linear and the equalized view

datashader.org: Common plotting pitfalls that get worse with large data

Take-aways

- Pixels are the bottleneck: aggregate, don’t draw every row
- Bin, summarize, smooth, plot: datashader does it with pixels as bins
- Decide where aggregation runs: on demand, precomputed, or in a query engine
- Check the pitfalls: look at both the linear and the equalized view